

CNN Accelerator 설계 보고서

MNIST INT8 추론 가속기 — Arty A7-100T / Vivado / Verilog

EEE351 Intelligent System Design and Application
Assignment 2 — Final Report (Individual)

전기전자공학과
김도현 (2022142223)
8팀

목차

Abstract	3
1 Introduction & Theory	3
1.1 서론	3
1.2 CNN 기본 연산과 타겟 네트워크	3
1.3 MLP / Fully-Connected의 분류	4
1.4 Line-buffer 기반 Sliding-Window 스캔	4
1.5 INT8 Quantization 이론	5
1.6 Winograd Convolution – 일반 이론	6
2 Problem Definition & Solution	6
2.1 문제 정의	6
2.2 Conv2를 최적화 1순위 할 – MAC의 90.2% 집중	7
2.3 DSP48E1 SIMD packing – 1 DSP에 INT8 곱 2개	7
2.4 DSP 분배	8
2.5 Inter-image pipelining과 분산 FSM 제어	10
2.6 AXI burst를 위한 32-bit 버퍼링 BRAM 설계	10
2.7 멀티클럭 사전 설계 (PS 100MHz / PL datapath + CDC)	10
2.8 Complex Winograd F(4×4, 3×3) 도입	10
2.9 Conv1 2× rebalance	11
3 Implementation – Baseline (INT8 Direct)	11
3.1 공통 시스템 공격 – 분산 데이터플로우 제어	11
3.2 오버클럭 이전 – Baseline 레이어 엔진	15
3.3 오버클럭 이후	20
4 Implementation – Winograd (제안 알고리즘)	20
4.1 Complex F(4×4,3×3) 데이터패스 구현	20
4.2 오버클럭 이후	22
5 Optimization Journey – Timing Closure	22
5.1 Baseline Track – 100 → 200 MHz (MET +0.011 ns)	22
5.2 Winograd Track – 200 MHz 시도 끝에 171.42 MHz로 met	26
5.3 두 트랙 종합	28
6 Results	29
6.1 Baseline – 오버클럭 이전: 기능 검증	29
6.2 Baseline – 오버클럭 이후: 보드 구현 & 실행	30
6.3 Winograd – 오버클럭 이전: bit-exact 검증	32
6.4 Winograd – 오버클럭 이후: util & 비교	33
7 Discussion	34
7.1 DSP48E1 SIMD packing의 재평가 – 특히 -128 보정	34
7.2 디버깅 사례 1 – AXI-Lite write hang	34
7.3 디버깅 사례 2 – NBA register race	35
7.4 Power 분석 (영세 우선순위 #2)	35
7.5 설계 통찰 – 병목의 이동과 분산 제어	35
7.6 한계와 Future Work	36
8 Conclusion	36
References	36
Appendix A. 코드 방췌체 (본문에서 참조)	38
Appendix B. CSR Memory Map & 칩부품	39

Abstract

본 프로젝트는 Arty A7-100T FPGA 위에서 MNIST 10,000장을 분류하는 INT8 CNN 추론 가속기를 설계하고, end-to-end latency 최소화를 목표로 최적화하였다. 검증 완료된 INT8 직접 컨볼루션 baseline은 200 MHz로 timing closure(WNS +0.011 ns)하여 보드에서 **10,000/10,000** 정확도인 약 **98 ms** latency를 실행하였다. 추가로, Conv2의 곱셈을 직접 conv 대비 3.13× 줄이는 **complex Winograd F(4×4, 3×3)** 및 DSP48E1 한 개로 두 INT8 곱을 처리하는 **SIMD packing** 을 제안·구현하였다. Winograd 데이터패스는 시뮬레이션에서 bit-exact로 검증되었고 implementation은 171.42 MHz로 timing-met이나, 보드 제출 마감 시점의 시간 제약으로 보드 실행은 수행하지 못해 1만 장 latency를 cycle 추정(78 ms)으로 보고한다.

1 Introduction & Theory

1.1 서론

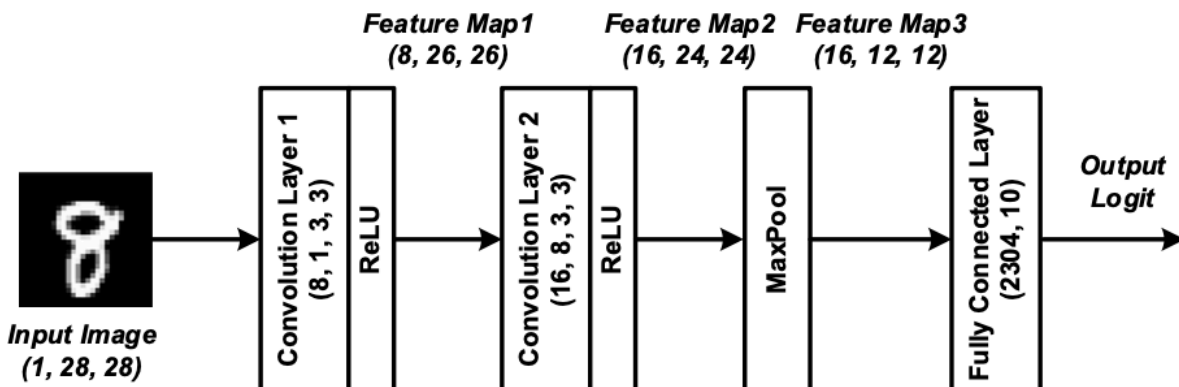
본 프로젝트의 목표는 MNIST 손글씨 숫자 10,000장을 분류하는 CNN 추론 가속기를 FPGA(Arty A7-100T) 위에 구현하는 것이다. 평가의 1순위는 10,000장 전체에 대한 **end-to-end latency** 최소화와, 그 다음으로 전력과 자원 사용량을 고려한다. 명세상 모든 신경망 연산은 PL(Programmable Logic)에서 수행되어야 하며, PS(MicroBlaze)는 메모리 read/write와 start/done 제어, 그리고 타이머 측정만 담당한다.

본 보고서가 다루는 세 가지는 가속기의 전체 아키텍처, 200 MHz timing closure에 이르는 문제해결 과정, 그리고 직접 컨볼루션 대비 곱셈을 줄이는 제안 알고리즘(complex Winograd)이다.

1.2 CNN 기본 연산과 타겟 네트워크

타겟 네트워크는 두 개의 컨볼루션 레이어, 하나의 max-pooling, 하나의 fully-connected 레이어로 구성된다. 각 컨볼루션은 3×3 커널, stride 1, zero-padding 없으므로 출력의 높이·너비가 입력보다 2씩 줄어든다. 데이터 흐름은 다음과 같다.

```
Input (1,28,28) -> Conv1 (8,1,3,3) -> ReLU -> (8,26,26)
                  -> Conv2 (16,8,3,3) -> ReLU -> (16,24,24)
                  -> MaxPool 2x2 -> (16,12,12)
                  -> Flatten (W->H->C) -> 2304
                  -> FC (2304->10) -> argmax -> class(0~9)
```



* Feature Map & Input Image Size : (Channel, Height, Width)

* Weight Size for Convolution Layer : (Output Channel, Input Channel, Kernel_Height, Kernel_Width)

* Weight Size for Fully Connected Layer : (Output Size, Input Size)

그림 1: 그림 F1: 타겟 CNN 구조도 (Input→Conv1→Conv2→MaxPool→FC→argmax)

각 연산의 정의는 표준 그대로 다. 2D 컨볼루션은 3×3 윈도우와 커널의 MAC(multiply-accumulate), ReLU는 음수를 0으로 클리핑, MaxPool은 2×2 영역의 최대값, FC는 2,304차원 입력과 (10×2304) 가중치의 행렬-벡터곱이다.

곱셈(MAC) 수를 레이어별로 보면 Conv1 48,672회(6.6%), **Conv2 663,552회(90.2%)**, FC 23,040회(3.1%)로, Conv2가 연산의 90% 이상을 차지한다.

1.3 MLP / Fully-Connected의 분류

마지막 FC 레이어는 MaxPool 출력 (16,12,12)을 width→height→channel 순으로 평탄화한 2,304차원 벡터를, (10×2,304) 가중치 행렬과 곱해 10개 클래스의 logit을 만든다. 그 중 최대값의 인덱스(argmax)가 예측 숫자다. 이는 다층 퍼셉트론(MLP의 한 층)의 형태이며, 본 가속기는 이 행렬-벡터곱과 argmax를 PL에서 수행한다.

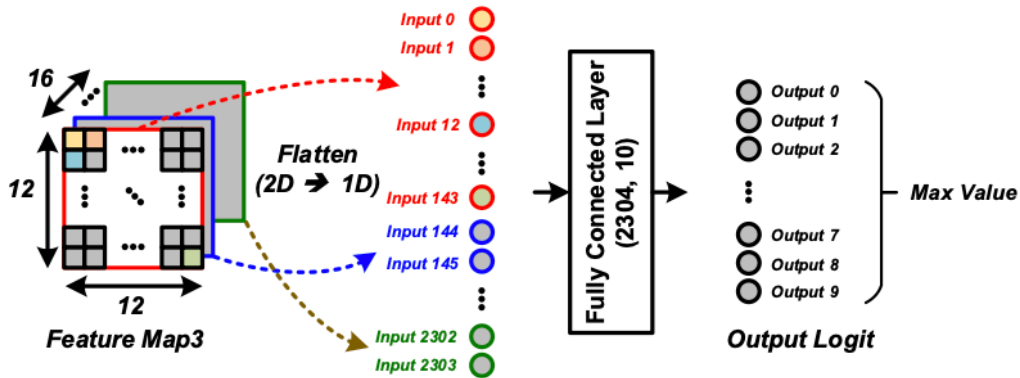
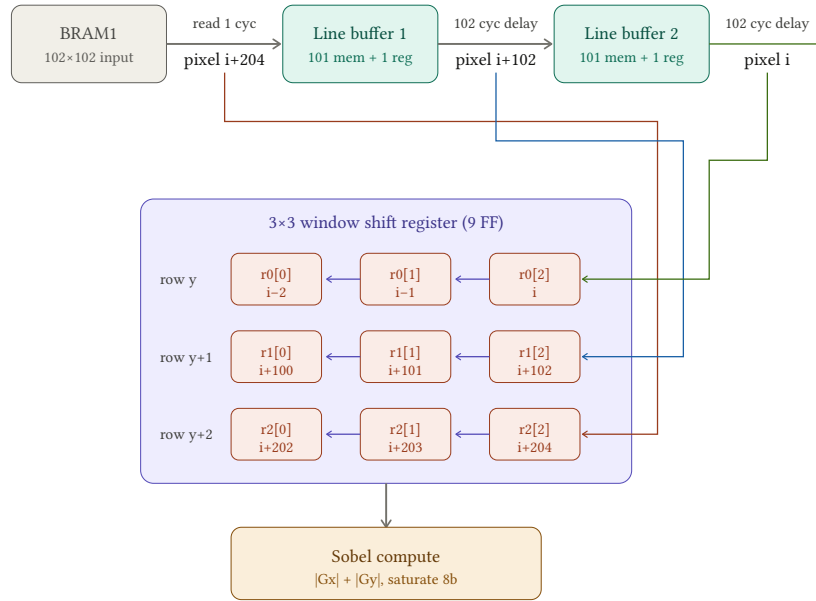


그림 2: 그림 F1b: MaxPool 출력의 flatten(W→H→C)과 FC 입력 구성

1.4 Line-buffer 기반 Sliding-Window 스캔

본 프로젝트의 컨볼루션 엔진은 입력 스트리밍에 line-buffer sliding-window 구조를 쓴다. 이는 이전 과제인 Sobel edge detection IP에서 검증된 구조를 재사용한 것이다.

BRAM에서 픽셀을 **raster-scan**(한 클럭에 한 픽셀)으로 읽고 이를 line buffer 여러 개에 직렬로 통과시키면, 매 클럭에 3×3 윈도우 하나가 완성된다(3×3 커널의 경우 line buffer 2개와 9개의 윈도우 레지스터가 필요하다). 앞단 윈도우가 완성되면 그 앞의 9개 탭은 윈도우 레지스터에 모두 동시에 가용하다.



새 픽셀은 각 row의 [2]에 수평 주입, 기존 데이터는 [2]→[1]→[0]로 한층 shift

그림 3: 그림 F6: Sobel 파이프라인의 윈도우 shift 데이터 재사용

이 구조의 중요한 점은 한 번에 출력 한 행(OH)씩 진행한다는 것이다. 따라서 여러 출력 행을 동시에 계산하는 OH 방향 병렬화는 line buffer를 갖거나 윈도우 레지스터를 복제하고 주소·valid 호직을 다중화해야 하므로 추가 비용이 든다. 이 비대칭은 Section 2.4에서 OH를 병렬 축으로 쓰지 않은 근거가 된다. 다만 이 비용은 윈도우를 만드는 단계에 대한 것이며, 이미 만들어진 윈도우 내부의 커널 축 선택과는 무관하다.

1.5 INT8 Quantization 이론

타겟 네트워크는 signed 8-bit 정수로 양자화되어 학습된 파라미터로 제공된다. 입력 이미지도 원래의 unsigned 8-bit(0~255)에서 signed 8-bit로 전처리된 값으로 주어진다.

추론 중 각 레이어의 누적은 오차를 줄이기 위해 비트를 확장한 정수(본 설계에서는 24-bit)로 수행하고, 출력 직전에 다음 규칙으로 다시 signed 8-bit로 절단한다: **LSB 10비트**를 산술 우측 시프트(**>>10**)한 뒤, 결과가 +127을 넘으면 127로, -128 미만이면 -128로 **saturation**하고, 하위 8비트를 출력으로 취한다. 컨볼루션 뒤에는 ReLU가 결합되어 하한이 0이 된다.

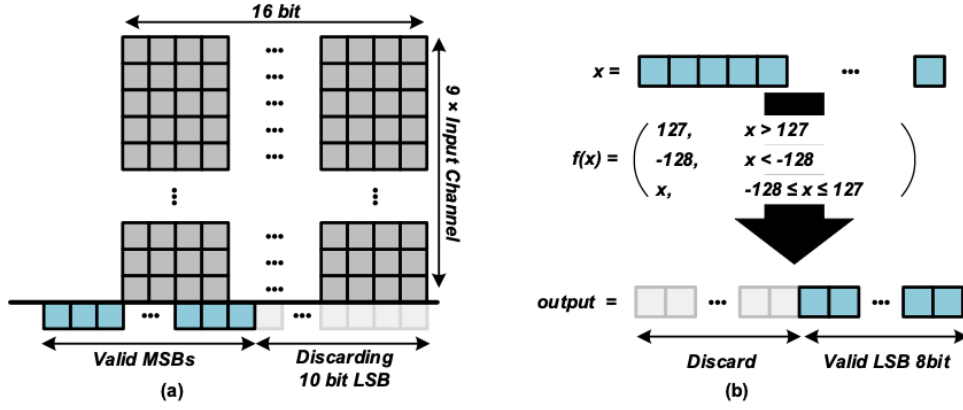


그림 4: 그림 F1c: 8-bit 양자화의 bit truncation-saturation (명세 규칙)

1.6 Winograd Convolution – 일박이론

Winograd minimal filtering은 컨볼루션을 다항식 곱셈으로 보는 데서 출발한다. 길이 r 의 커널과 길이 n 의 입력 신호를 각각 다항식 $g(x)$, $d(x)$ 로 보면, 1D 컨볼루션의 출력은 곱 다항식 $s(x) = g(x) \cdot d(x)$ 의 계수값 정확히 같다. g 가 $r-1$ 차, d 가 $n-1$ 차이므로 s 는 $(n+r-2)$ 차, 즉 계수가 $n+r-1$ 개다.

차수 D 의 다항식은 서로 다른 $D+1$ 개의 점에서의 값으로 유일하게 결정된다(보간 정리). 따라서 $s(x)$ 의 $n+r-1$ 개 계수를 얻으려면, 다항식을 직접 전개(직접 컨볼루션 = $n \cdot r$ 곱셈)하는 대신 서로 다른 $n+r-1$ 개 점 α_i 에서 $g(\alpha_i) \cdot d(\alpha_i)$ 를 곱한 뒤 나그랑주 보간으로 계수를 복원하면 된다. 이때 곱셈은 점마다 한 번씩, 총 $n+r-1$ 회만 일어난다. 점에서의 평가($g(\alpha_i)$, $d(\alpha_i)$)와 보간(복원)은 모두 입력·커널의 선형 결합이므로 상수 행렬과의 곱(덧셈·시프트)으로 처리된다 – 곱셈기 비용을 가산기로 옮기는 것이 Winograd의 핵심이다. 이것이 일박이론으로 $F(m, r)$ ($m = n - r + 1$ 출력)을 $m \cdot r$ 대신 $m + r - 1$ 회 곱셈으로 계산하는 Toom-Cook 알고리즘이다.

여기에 한 가지 절약이 더 있다. 보간점 중 하나로 무한대 점(∞)을 도입하면(최고차 계수를 직접 취하는 것에 대응) 유한점을 하나 덜 써도 되고, 모듈러 산술로 보면 필요한 점 수를 $n+r-1$ 보다 줄일 수 있다(Toom-Cook). 본 프로젝트가 쓰는 점 집합에도 ∞ 가 포함된다.

행렬 형태로 쓰면 2D 변환은 다음과 같다.

$$Y = A^T [(GgG^T) \circ (B^T dB)] A$$

g 는 커널, d 는 입력 타일이며, G (weight 평가)· B^T (input 평가)· A^T (보간/복원)는 선택한 보간점들로부터 유도되는 상수 행렬이다. 곱셈은 element-wise 곱($\circ$)에서만 발생한다. 예로 1D $F(2, 3)$ 은 점 $\{0, 1, -1, \infty\}$ 를 써서 직접 6회 대신 4회 곱셈으로 두 출력을 낸다. 2D $F(m \times m, 3 \times 3)$ 은 직접 $m^2 \cdot 9$ 대신 $(m+2)^2$ 회 곱셈을 쓰므로, m 을 키웁수록 절감이 커진다 – $F(2, 3)$ 은 $9 \rightarrow 4(2.25 \times)$, $F(4, 3)$ 은 $36 \rightarrow 16$ per output로 $144 \rightarrow 36(4 \times)$.

그러나 표준 실수 보간점으로 m 을 키우면 변환 행렬에 점점 복잡한 분수가 나타난다. 예컨대 $F(4, 3)$ 의 G 에는 $1/24$ 같은 원소가 들어간다. 이 분수는 INT8 정수 격자타어긋나 양자화 정밀도를 떨어뜨리고, 분모를 흡수하기 위한 추가시프트·자릿수가 DSP·자원 예산을 더 소비한다. 이 두 문제가 Section 2.8에서 복소수 보간점으로 확장하는 동기가 된다.

2 Problem Definition & Solution

2.1 문제 정의

본 프로젝트의 목표는 MNIST 10,000장을 Arty A7-100T 위에서 분류하되 **end-to-end latency**를 최소화하는 것이다. 제약은 정확도 무손실(제공된 INT8 파라미터를 비트-정확하게 재현)과 보드 자원 한도(DSP 240, BRAM 135, LUT 63K, FF 126K)다. 평가 우선순위는 latency > power > 자원이며, 모든 설계 선택은 논리적으로 정당화되어야 한다. 이하의 결정들은 이 목표·제약에서 차례로 도출된다.

2.2 Conv2를 최적화 1순위로 - MAC의 90.2% 집중

먼저 어디에 노력을 집중할지를 정해야 한다. Section 1.4 앞의 MAC 분포를 보면 곱셈 연산은 Conv1 48,672 회(6.6%), **Conv2 663,552회(90.2%)**, FC 23,040회(3.1%)로, Conv2가 전체의 90% 이상을 차지한다. 따라서 자원 배분과 알고리즘 최적화(후술하는 Winograd)는 모두 Conv2에 집중한다. FC나 Conv1을 먼저 최적화하는 것은 전체 latency 기여가 작아 투자 대비 효과가 낮으므로 택하지 않았다.

2.3 DSP48E1 SIMD packing - 1 DSP에 INT8 곱 2개

DSP 분배(Section 2.4)는 1개의 DSP가 곱셈을 1개 하느냐 2개 하느냐에 따라 기준 자체가 달라지므로, packing을 먼저 정한다.

Artix-7의 DSP48E1은 25×18 signed multiplier다. INT8 곱을 1개씩만 수행하면 25비트 A 포트가 낭비된다. 대신 두 weight W_0, W_1 과 공유 activation X 를 다음과 같이 한 곱에 실는다.

$$A = W_1 \cdot 2^{17} + W_0 \quad (25\text{-bit}), \quad B = X \quad (18\text{-bit})$$

$$A \cdot B = (W_1 \cdot X) \cdot 2^{17} + (W_0 \cdot X)$$

[Structure of DSP48E1]

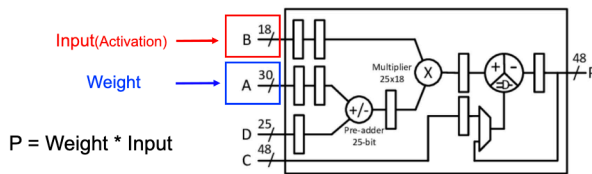


그림 5: 그림 F7: DSP48E1 구조 (A 25b / B 18b / 25×18 mult / P 48b) 그림 6: 그림 F8: SIMD packing 비트맵 (W0 하위 17b, guard, W1 상위)

그림 F8이 보여주듯, $W_0 \cdot X$ 는 하위 17비트(guard bits 포함)에, $W_1 \cdot X$ 는 그 위에 자리하여 둘이 겹치지 않는다. 따라서 한 번의 DSP 연산에서 다음과 같이 분리 추출한다.

$$\begin{aligned} P_0 &= W_0 \cdot X = \text{sint17}(P \bmod 2^{17}) \\ P_1 &= W_1 \cdot X = \text{sint16}(\text{floor}(P / 2^{17}) \bmod 2^{16}) + [P_0 < 0] \cdot 256 \cdot X \cdot \text{ovf} \\ &\quad (\text{ovf} = [W_1 = -128 \text{ and } W_0 < 0]) \end{aligned}$$

이론상 $W_1 = -128$ 이고 $W_0 < 0$ 일 때 25비트 표현이 overflow하므로 산술 보정항 $-256 \cdot X \cdot \text{ovf}$ 과 carry 보정 $[P_0 < 0]$ 을 두어 bit-exact를 보장한다. 다만 본 사례의 실제 제공 가중치에는 **-128**이 없다. Python으로 세 레이어 weight 범위를 전수 확인한 결과 Conv1 $[-127, 123]$, Conv2 $[-127, 127]$, FC $[-127, 118]$ 로 **-128** 개수가 모두 **0**이며, “SIMD packing overflow condition does NOT occur”임을 검증했다.

Layer	Shape	Total	Min	Max	-128 cnt	Result
Conv1	(8,1,3,3)	72	-127	123	0	PASS
Conv2	(16,8,3,3)	1152	-127	127	0	PASS
FC	(10,2304)	23040	-127	118	0	PASS

[RESULT] All weights are in range [-127, +127]. → No Overflow

No weight equals -128 in any layer.

SIMD packing overflow condition does NOT occur.

그림 7: 그림 F9: weight -128 부재 검증 - 레이어별 min/max, -128 cnt=0, “No Overflow”

따라서 이 보정 경로는 본 과제에서 실제로 동작하지 않으며, 그 값은 실제 성능이 아니라 입의 INT8 가중치에 대한 일관성에 있다(자세한 위치 설정은 Section 7.1).

여기서 한 가지 걱정이 더 따라온다: 무엇을 패킹할 것인가. Weight에는 -128이 없지만, **Conv1** 출력 **activation**에는 -128이 있을 수 있다. 만약 activation을 패킹한다면 -128 처리용 추가 로직이 PE마다 필요해 비용이 커진다. 따라서 weight 두 개(W_0, W_1)를 패킹하는 것이 최선이다. 그 러므로, 한 DSP 묶음에서 나오는 두 결과가 서로 다른 출력 채널(OC) 두 개가 되도록 강제된다(OC 방향 packing). 이 사실은 Section 2.4에서 “왜 IC가 아니라 OC를 2× 방향으로 펼치는지”의 직접 근거가 된다.

이 기법으로 DSP당 throughput이 2배가 된다 – 같은 DSP 예산으로 두 배의 곱셈을 수행한다. 선행연구 대비 차별점은, Xilinx WP486은 DSP48E2(27비트 A 포트) 전용이고 Vestias(FPL’17)는 -128에서 손상이 발생 하는 반면, 본 기법은 **DSP48E1(25비트)**에서 산술 보정만으로 전 **INT8** 케이스를 무손상 처리 한다는 점 이다(전수 검증은 Section 6).

2.4 DSP 분배

각 레이어에 DSP를 몇 개 줄 것인가는 곱셈 수에서 출발한다. 각 레이어의 연산을 중첩 루프로 정의하고, 곱셈 수를 세고, 곱셈 수에 비례해 240개를 나눈 이상적 (비정수) 값을 구한 뒤, 정수 제약·약수 제약·packing 방향(Section 2.3)·구조 단순성을 차례로 적용해 실제 정수 분배로 좁힌다.

2.4.1 각 레이어의 연산 정의

각 conv 레이어가 어떤 곱셈을 몇 번 하는지를 중첩 루프로 정의한다. 이 루프의 각 차원(KH, KW, IC, OC, OH, OW)이 곧 병렬화 후보 축이며, 어떤 축을 병렬화하든 그 병렬도는 해당 차원의 약수 여야 한다(나누어떨어지지 않으면 잔여 처리 로직이 붙는다). Conv2를 대표로 보이던 다음과 같다.

```
# Conv2: IC=8, OC=16, KH=KW=3, OH=OW=24
for oh in 0..23:
  for ow in 0..23:
    for oc in 0..15:
      acc = 0
      for ic in 0..7:
        for kh in 0..2:
          for kw in 0..2:
            acc += act[ic][oh+kh][ow+kw] * w[oc][ic][kh][kw]
      out[oc][oh][ow] = truncate_relu(acc) # >>10, saturate, ReLU
```

Conv1(IC=1, OC=8, K=3×3, OH=OW=26)과 FC(2,304→10의 행렬-벡터곱)도 같은 형식의 의사코드로 제시 한다.

2.4.2 곱셈 수와 이상적 DSP 비례 분배

위 루프로부터 곱셈 수가 정해진다: Conv1 48,672, Conv2 663,552, FC 23,040, 총 735,264회다. 모든 레이어가 같은 시간에 처리를 마쳐 파이프라인이 균형을 이루려면, 곱셈 수에 비례해 **DSP**를 나누는 것이 이상적이다(packing의 2배 throughput은 모든 레이어에 공통이므로 비율에는 영향을 주지 않는다). DSP 240개를 모두 쓴다고 가정하면 레이어별 이상적 DSP 값은 다음과 같다.

레이어	곱셈 수	비율	이상적 DSP	실제 분배
Conv1	48,672	6.6%	15.9	18 (→ Wino 36)
Conv2	663,552	90.2%	216.6	192
FC	23,040	3.1%	7.5	16

표 1: 표 T8: 곱셈 수 → 비율 → 이상적 DSP → 실제 분배

FC는 약수·병렬화 구조상 16 DSP로 구현되어 이상값 7.5보다 크다. baseline 총량은 $192 + 18 + 16 = 226/240$ (94%)이다. 문제는 이상값이 정수가 아니라는 점이다. 실제로는 (a) DSP가 정수여야 하고, (b) 병렬도가 각 축의 약수여야 하며, (c) packing이 OC 방향 2×를 강제하고(Section 2.3), (d) 누적·제어 구조가 단순할수록 좋다. 이 제약들을 Conv2에 적용한 결과가 다음 항이다.

2.4.3 Conv2: 144 기각, 192 확정

본 보고서의 축명명은 RTL(`conv2_engine.v`)을 따른다.

- 언롤(공간 병렬) = $K_row(\text{커널 행, } KH=3) - pe_x[K_row][IC]$ 인덱싱, PE array는 $192 = OC_pair8 \times IC8 \times K_row3$.
- 시퀀셜 누적 = $K_col(\text{커널 열, } KW=3) - fsm_col_sel$ 이 한 열씩 선택하고 `kcol_accumulator`가 3사이클에 걸쳐 누적.

즉 커널 행은 공간적으로 펼쳐지고(병렬), 커널 열은 3사이클에 걸쳐 시분할 누적한다.

직접 Conv2의 한 출력 픽셀은 $IC8 \times KH3 \times KW3 = 72$ MAC을 요구하고, 전체는 $72 \times OC16 \times OH24 \times OW24 = 663,552$ 이다. 한 사이클에 처리할 곱셈 수의 두 후보는 다음과 같다.

$$\begin{aligned} 144 &= IC8 \times K9 \times SIMD2 && (IC \text{ 전체} \times \text{커널 9탭 전부 언롤} \times \text{packing 2}) \\ 192 &= OC_pair8 \times IC8 \times K_row3 && (OC16 \text{ packing 2} \times IC \text{ 전체} \times \text{커널 '행'만 언롤}) \end{aligned}$$

144를 기각한 이유는 이상값에 있다. Conv2의 이상적 몫은 216.6 DSP인데, 144만 쓰면 약 70여 개의 DSP가 남는다. 남은 자원만큼 사이클이 길어져 파이프라인 균형이 깨지므로, 한 축의 병렬도를 키워 240 이상에 더 가깝게 채워야 한다.

확정한 분배는 **192 DSP** ($= OC_pair8 \times IC8 \times K_row3$, Conv2 몫)이다. 이는 Conv2의 이상값 216.6에 144보다 훨씬 가까우면서(Conv1 18-FC 16라 함치면 전체 226/240, 94%를 채운다) 동시에 packing이 쉽고 누적 구조가 단순한 방향이다. 세 가지 세부 결정이 여기에 얹혀 있다.

첫째, 왜 OC 방향 2×인가(IC 패킹이 아니라). Section 2.3에서 본 대로 한 DSP 묶음의 두 결과는 서로 다른 OC가 되도록 *weight*를 패킹해야 한다. IC를 패킹 축으로 쓰려면 *activation*을 패킹해야 하는데, Conv1 출력 activation에는 -128이 있을 수 있어 PE마다 보정 로직이 붙는다. 따라서 OC를 packing 축으로 택했다.

둘째, 왜 커널 열(K_col)을 펼치지 않고 3사이클 누적하는가. 한 윈도우의 $3 \times 3 = 9$ 탭은 `window_register`에 9개가 모두 동시 가용하므로, 원리상 9탭을 전부 펼치는 것도 가능하다(이는 곧 144 정도). 그러나 9탭 전부 언롤은 240 이상을 다 쓰지 못하고, 반대로 OC·IC를 더 펼치면 예산을 초과한다. 240 안에서의 균형점이 바로 “커널 행 3개만 공간 언롤 + 커널 열 3개는 3사이클 시퀀셜 누적”이다. 이 경우 누적 경로가 `krow_ic_adder_tree`(행·IC 합) → `kcol_accumulator`(열 3사이클 합)로 단순하게 떨어진다. 이때 한 이미지의 순수 연산 사이클은 $OH24 \times OW24 \times KW3 = 1,728$ 이고, 여기에 파이프라인 fill/drain 오버헤드가 더해져 실제 throughput floor는 약 **1,799 cyc/img**가 된다(Section 5에서 이 값을 latency 추정에 사용).

셋째, 왜 IC나 OC를 더 작게 쪼개지 않는가. OC나 IC를 부분 그룹으로 더 나누면 부분합을 따로 보관했다가 나중에 더하거나 time-multiplexing해야 하므로 누적·스케줄 FSM이 복잡해진다. IC=8 전체를 한 번에 펼치면 cross-IC 합이 하나의 가산 트리로 끝나 제어가 간단하다.

병렬화 옵션	DSP	기각/채택 사유
144 = 9탭 전부 언롤	144	기각 - 예산 미달(70 DSP 남음)
IC를 패킹 축으로	—	기각 - activation(-128) 패킹 추가 로직
OC·IC 추가 분할	>240	기각 - 예산 초과 + 부분합 FSM 복잡
커널 열도 언롤	—	기각 - 9탭 언롤=144 경로와 동일
행 언롤 + 열 3-cyc 누적	192	채택 - 예산 근접·단순 누적

표 2: 표 T9: 병렬화 옵션 vs DSP 예산·packing·누적 단순성

주: Section 1.4의 line-buffer 비대칭은 윈도우를 만드는 단계(여러 output row를 동시 생성 시 line buffer 복제 비용)에 대한 것이지, 이미 만들어진 윈도우 내부의 9탭 선택과는 무관하다. 윈도우 내부 축(K_row/K_col) 선택의 근거는 위의 DSP 예산 트레이드오프이고, Section 1.4는 OH(출력 행) 병렬화를 쓰지 않은 이유로만 인용한다.

2.5 Inter-image pipelining과 분산 FSM 제어

1만 장을 연속으로 추론할 때는 단일 이미지 latency보다 stage 간 overlap이 throughput을 좌우한다. 따라서 stage 사이를 ping-pong BRAM 버퍼로 분리하여, 한 engine이 다음 이미지를 쓰는 동안 다음 engine이 이전 이미지를 읽도록 했다. 제어는 중앙 컨트롤러를 두지 않고, 각 engine이 자체 FSM과 bank-toggle FF를 가지며 stage 간 write_done / read_done (각 1-cycle pulse) 핸드셰이크로만 동기화한다.

중앙 컨트롤러(단일 거대 FSM) 방식은 제어 신호가 모든 engine으로 fanout되어 fanout-timing closure에 불리하고, 특히 오버클럭 시 병목이 된다(실제로 Section 5의 closure에서 high-fanout 제어 net이 반복적으로 문제였다). 분산 제어는 이 부담을 구조적으로 회피한다 – 이 이전의 정량 평가는 Section 7.5에 둔다.

2.6 AXI burst를 위한 32-bit 비대칭 BRAM 설계

PS→PL 데이터 전송은 전체 latency에서 큰 비중을 차지한다(실측에서 CDMA feed가 72%, Section 7.6). 따라서 data-path BRAM을 **Port A는 32-bit(PS write, AXI burst 친화), Port B는 엔진 소비 폭(8~128-bit)**의 비대칭 구성으로 설계하여, PS 측은 32-bit word-burst로 전송 효율을 높이고 엔진 측은 필요한 폭으로 병렬 read하게 했다.

대안으로 대칭 8-bit 포트는 전송이 비효율적이고, .coe로 BRAM을 초기화하는 방식은 명세가 급지할 뿐 아니라 PS 전송 시간을 측정에서 누락시키므로 모두 기각했다. 모든 weight-image는 PS가 AXI로 write한다.

2.7 멀티클럭 사전 설계 (PS 100MHz / PL datapath + CDC)

오버클럭은 PL datapath에서만 의미가 있다. 처음부터 단일 클럭으로 설계하면 클럭을 쏠 때 PS-AXI까지 끌려 올라가 closure가 불가능하다. 따라서 PS(100MHz)와 PL datapath 클럭을 처음부터 분리하고, 경계에 CDC(cdc_pulse_sync / cdc_bit_sync)를 끼어 끼어 두어 후일 PL만 150/200MHz로 쏠릴 여지를 확보했다. 이 사전 분리가 Section 5 오버클럭 로드맵 전체의 전제다(구현 동작은 Section 3). 단일 클럭 도메인은 오버클럭 확장성이 없고 나중에 재설계 비용을 치르게 되므로 택하지 않았다.

2.8 Complex Winograd F(4×4, 3×3) 도입

Conv2가 곱셈의 90%를 차지하므로, 곱셈 수 자체를 줄이는 것이 가장 효과적이다. Section 1.6에서 보았듯 $F(m, 3)$ 의 출력당 곱셈은 $9 \rightarrow (m+2)^2/m^2$ 로 줄고, m 이 클수록 절감이 크다. 따라서 원리상 가장 좋은 선택은 m 을 키우는 것이다.

그러나 표준 실수 보간점으로 m 을 키우면 두 문제가 동시에 악화된다. 변환 행렬 원소는 라그랑주 보간계수이고, 그 분모에는 보간점들 사이의 거리 곱이 나타난다 – 점이 많아지고 서로 멀어질수록 분모가 큰 행성수가 되어 행렬에 복잡한 분수가 생긴다. 그 결과 (1) 분수가 INT8 정수 격자란 어긋나 양자화 정밀도가 추가로 떨어지고(baseline이 이미 93% 수준이라 여유가 적다), (2) 분모를 흡수하기 위한 자릿수 확장이 DSP·자원 예산을 더 소비한다.

핵심 관찰은 분모가 보간점들 사이 거리의 곱이라면, 점들을 서로 가깝게 두면 분수를 통제할 수 있다는 것이다. 점을 실수축에만 두면(예: $0, \pm 1, \pm 2, \dots$) 점이 늘수록 점 사이 거리가 커질 수밖에 없지만, 복소 평면으로 확장하면 작은 거리를 유지한 채 점을 더 모을 수 있다. 유한점 $\{0, \pm 1, \pm i\}$ 를 보면 서로 간 거리가 $1, \sqrt{2}, 2$ 세 값뿐이어서 분모가 작은 정수로 유지된다. 여기에 무한대 점 ∞ 를 더한 $\{0, \pm 1, \pm i, \infty\}$ 가 이렇게 거리를 통제하며 모을 수 있는 최대 집합(6개)이고, 이 6점으로 만들 수 있는 변환은 $F(4, 3)$ 이 한계다($m+2=6$). $F(5, 3)$ 은 점이 7개 필요한데 거리를 작게 유지하며 7번째 점을 추가할 수 없어 사실상 불가능하다 – 가능했다면 더 키웠을 것이다. 이 6점을 택하면 G, B^T, A^T 가 모두 Gaussian integer($\{0, \pm 1, \pm i, \pm 4\}$)가 되어 분수가 전혀 없고, $1/16$ 스케일만 출력 시프트로 흡수된다.

$$\begin{aligned}
B^\top &= \begin{bmatrix} 4 & 0 & -5 & 0 & 1 & 0 \\ 0 & -4 & -4 & 1 & 1 & 0 \\ 0 & 4 & -4 & -1 & 1 & 0 \\ 0 & -2 & -1 & 2 & 1 & 0 \\ 0 & 2 & -1 & -2 & 1 & 0 \\ 0 & 4 & 0 & -5 & 0 & 1 \end{bmatrix} & G &= \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & i & -1 \\ 1 & -i & -1 \\ 0 & 0 & 1 \end{bmatrix} & A^\top &= \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & i & -i & 0 \\ 0 & 1 & 1 & -1 & -1 & 0 \\ 0 & 1 & -1 & -i & i & 1 \end{bmatrix} \\
G &= \begin{bmatrix} \frac{1}{4} & 0 & 0 \\ -\frac{1}{6} & -\frac{1}{6} & -\frac{1}{6} \\ -\frac{1}{6} & \frac{1}{6} & -\frac{1}{6} \\ \frac{1}{24} & \frac{1}{12} & \frac{1}{6} \\ \frac{1}{24} & -\frac{1}{12} & \frac{1}{6} \\ 0 & 0 & 1 \end{bmatrix} & A^\top &= \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & 2 & -2 & 0 \\ 0 & 1 & 1 & 4 & 4 & 0 \\ 0 & 1 & -1 & 8 & -8 & 1 \end{bmatrix} & B^\top &= \begin{bmatrix} 4 & 0 & 0 & 0 & -4 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & -1 & 1 & -1 & 1 & 0 \\ 0 & -i & -1 & i & 1 & 0 \\ 0 & i & -1 & -i & 1 & 0 \\ 0 & -4 & 0 & 0 & 0 & 4 \end{bmatrix}
\end{aligned}$$

그림 8: 그림 F5a: 표준(실수) $F(4,3)$ 변환행렬 $\pm 1/24$ 분수 포함 그림 9: 그림 F5b: complex $F(4 \times 4, 3 \times 3)$ 변환행렬 - 전 부 Gaussian integer

복소 확장은 곱셈 수에 손해도 준다. 복소 곱은 본래 실수 곱 4회지만, 실수 입력의 켤레 대칭 $s(-i) = \overline{s(i)}$ 로 켤레 전의 절반이 공짜로 얻어지고, 남은 복소 곱은 Gauss 트릭($k_1 = a(c+d), k_2 = c(b-a), k_3 = d(a+b)$)으로 1회를 실수 곱 3회로 줄인다.

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i$$

$$\begin{aligned}
k_1 &= a(c + d) & \text{Re} &= k_1 - k_3 \\
k_2 &= c(b - a) & \text{Im} &= k_1 + k_2 \\
k_3 &= d(a + b)
\end{aligned}$$

그림 10: 그림 F5c: Gauss 트릭 - 복소 곱 1회를 실수 곱 3회로

그 결과 한 타일당 곱셈은 (real,real) 16 + (real,cplx) 4×3 + (cplx,real) 4×3 + (cplx,cplx) 2×3 = **46**회 가 된다. 전체로는 직접 conv의 663,552회가 211,968회로 **3.13×** 감소 한다 - 실수 $F(4, 3)$ 의 36회(4×)보다는 많지만, 그 4×는 INT8에서 정밀도·예상 손실을 동반하므로 취할 수 없다. **bit-exact**를 유지하면서 얻을 수 있는 최대 절감치 **complex $F(4, 3)$ 의 3.13×**이며, 이것이 본 사례의 정확도 무손실 제약 아래 최선이다.

2.9 Conv1 2× rebalance

Conv2를 Winograd로 빠르게 만들면 병목이 Conv1로 이동한다. 따라서 Conv1을 DSP 18→36으로 늘리고 2-round를 1-round로 바꿔 약 837 cyc로 재균형했다. Conv1을 방치하면 Winograd의 3.13× 이득이 Conv1에 가려져 무의미해지므로, rebalance는 선택이 아니라 필수다. 병목 이동의 정량 분석은 Section 7.5에 있다.

3 Implementation — Baseline (INT8 Direct)

본 장은 Section 3 직전(§2)에서 내린 설계 결정을 RTL·블록 디자인·메모리 맵·FSM으로 어떻게 실현했는지에 집중한다. “왜 이 구조인가”는 반복하지 않고 §2의 해당 항목을 가리킨다. 구성은 공통 시스템 공격(Section 3.1), 레이어별 엔진(Section 3.2), 그리고 오버클럭 이후(§5 위임) 순이다.

3.1 공통 시스템 공격 — 분산 데이터플로우 제어

Baseline과 Winograd가 공유하는 공격이다. 설계 결정과 근거는 Section 2.5 Section 2.7에 있다.

3.1.1 PS-PL 블록 구성

전체 시스템은 Vivado Block Design 상에서 PS 영역(MicroBlaze + local memory), AXI Interconnect, 메모리 접근용 AXI BRAM Controller, 이미지 feed용 AXI CDMA, 제어용 CSR(AXI-Lite slave), 클럭 생성용 Clocking Wizard, DRAM 접근용 MIG(DDR3), 그리고 가속기 본체 `cnn_accelerator` (PL)로 구성된다. 형제 제약대로 모든 신경망 연산은 PL에서 수행되고, PS는 메모리 read/write와 start/done 제어, 타이머 측정만 담당

한다. MNIST 10,000장 테스트셋은 빅드 시 C 헤더 배열로 DRAM에 적재되며, MicroBlaze가 이를 읽어 AXI 경로를 PL의 입력 BRAM에 전달한다.

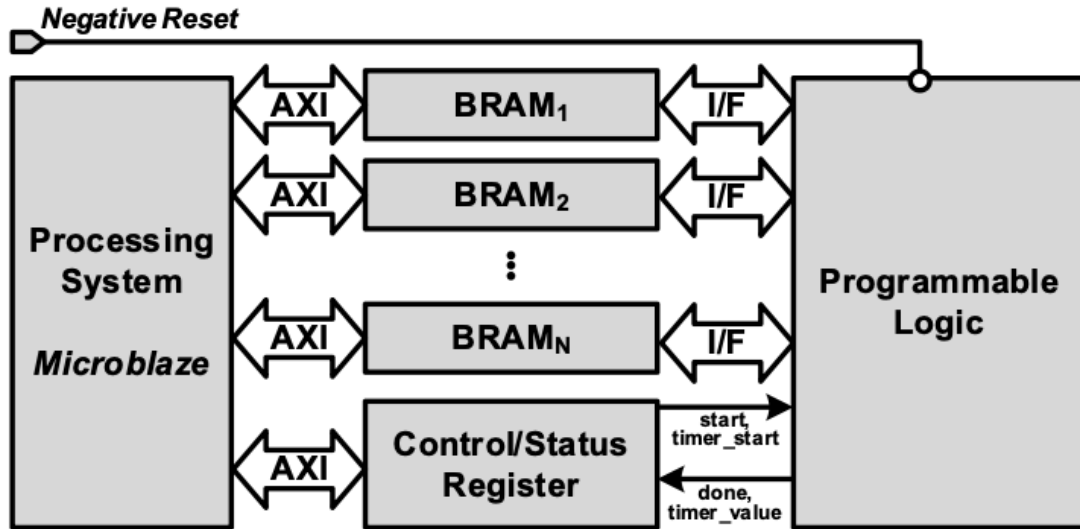


그림 11: PS-PL 시스템 블록 다이어그램 (전체 경선은 첨부 PDF의 F2 참조)

PL 본체 `cnn_accelerator`는 다음을 외부로 노출한다 - (1) PS가 write하는 4개 BMG(Input / Conv1 weight / Conv2 weight / FC weight)의 Port A, (2) PS가 read하는 결과 BMG(Output)의 Port B, (3) CSR과 주고받는 제어 신호 `enable/start/img_ready` (PS→PL)와 `img_done/input_consumed` (PL→PS)다. 이 중 stage 사이 ping-pong 버퍼(Input / C1C2 / C2Pool / PoolFC)와 결과 BMG는 본체가 직접 인스턴스화하고, 레이어별 weight BMG는 각 엔진 내부에 인스턴스화하여 Port A와 본체를 passthrough한다 - weight 경선을 엔진에 응집해 top-level 배선을 단순화한 것이다. FC weight는 512-bit폭이라 32-bit MicroBlaze가 write 하려면 **32→512 datawidth converter**와 512-bit AXI BRAM Controller를 경유한다(Section 2.6).

.coe로 BMG를 초기화하는 것이 명세상 금지되므로 모든 weight-image는 PS가 C 헤더 배열에서 AXI를 통해 BRAM으로 전송한다. 멀티클럭은 Section 2.7의 결정대로, PS-AXI-CSR는 100 MHz(`ac1k`), datapath는 별도 클럭(`clk`)으로 분리하고 두 도메인 경계 신호를 CDC 동기화기로 건넨다. 클럭 분리·CDC의 구현과 동작은 오버클럭 로드맵의 전제이므로 상세는 Section 5에 둔다.

3.1.2 메모리 구조 - 비대칭 폭 BRAM과 ping-pong 버퍼

모든 데이터 이동은 Block Memory Generator(BMG) IP로 실현된다. 각 BMG는 **Simple Dual Port(SDP)** 구성으로 write 포트(Port A)와 read 포트(Port B)를 분리하며, stage 사이 버퍼는 2 bank ping-pong으로 둔다 - 한 engine이 한 bank에 다음 이미지를 쓰는 동안 다음 engine이 다른 bank에서 이전 이미지를 읽는다. bank 선택은 각 엔진 내부의 toggle FF가 관리하고, 물리 주소의 MSB로 prepend된다.

핵심은 비대칭 폭(**asymmetric width**) 설계다(근거는 Section 2.6). PS가 접근하는 두 끝단 BMG(Input, Output)는 Port A/B의 폭이 다르다.

- **Input BRAM (bram_input)**: Port A = 32-bit × 512 word (PS가 AXI burst로 4 byte/cycle write), Port B = 8-bit × 2048 (Conv1이 픽셀 단위 byte read). 총 2 KB = 2 bank × 1024 byte. Vivado asymmetric BMG의 기본 **little-endian** 매핑에 따라 Port A의 word k 가 byte $4k, 4k\{+1\}, 4k\{+2\}, 4k\{+3\}$ 를 담고, Port B의 주소 $4k\{+j\}$ read는 word k 의 j 번째 byte를 돌려준다. 한 장(28×28 = 784 byte)은 정확히 196 word로 나누어 떨어져 padding이 필요 없다.
- **Output BRAM (bram_output)**: Port A = 8-bit × 16384 (PL이 `img_done`마다 분류 결과 1 byte 누적 write), Port B = 32-bit × 4096 (PS가 종런 후 4 결과/word로 burst read). per-image read를 없애 파이프라이닝 중 결과 손실을 막는다.

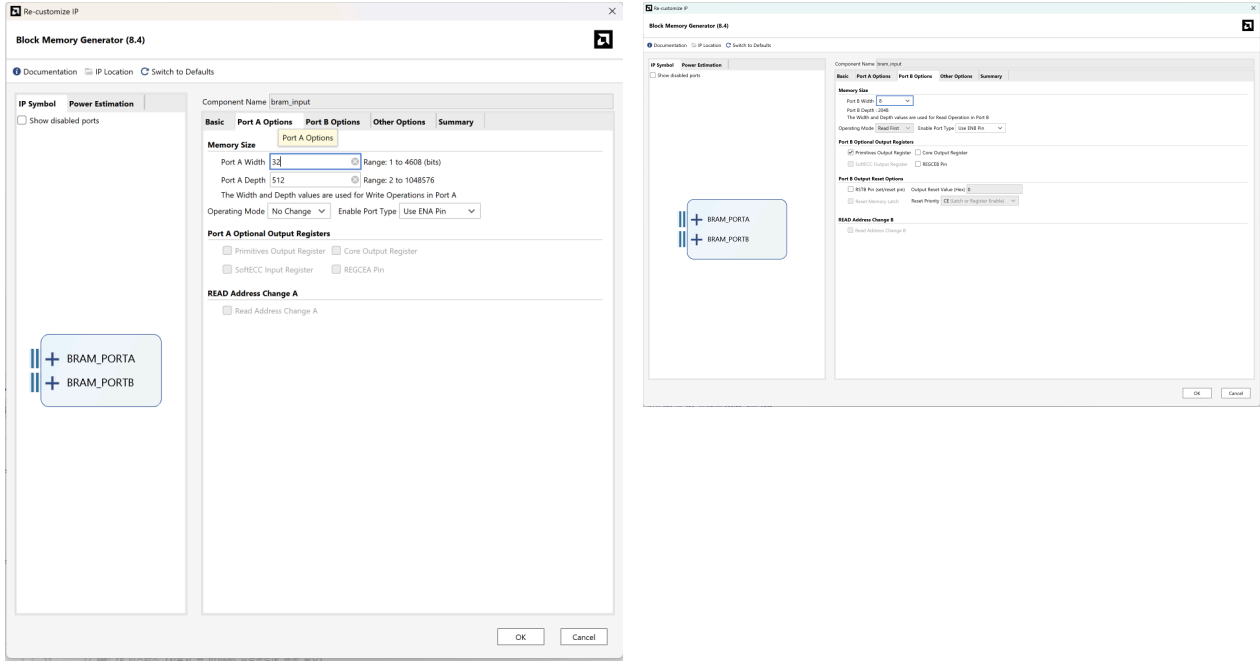


그림 12: bram_input의 비대칭 포트 구성 – (좌) Port A Width 32 / Depth 512 = PS의 AXI burst write 측, (우) Port B Width 8 / Depth 2048 = Conv1의 byte read 측. 같은 메모리측 PS는 32-bit word를, Conv1은 8-bit byte를 접근한다.

stage 7의 ping-pong 버퍼와 weight BMG의 구성을 따로 정리한다. (read latency L 은 현재 구성값이며, $L=2$ 채택의 타이밍 근거는 Section 5과 부록에 둔다.)

BMG	Port A (write)	Port B (read)	Depth	L	write → read
bram_input	32b	8b	512 / 2048	2	PS → Conv1
bram_c1_to_c2	64b (byte-we)	64b	2048	2	Conv1 → Conv2
bram_c2_to_pool	128b	128b	2048	2	Conv2 → MaxPool
bram_pool_to_fc	128b	128b	512	2	MaxPool → FC
bram_output	8b	32b	16384 / 4096	1	PL → PS

표 3: 데이터-패스 BMG (ping-pong + 입출력). Port 폭이 다른 bram_input·bram_output이 비대칭 구성이다.

weight BMG	폭	Depth(사용)	적재 시점	소비
conv1_weight_bram	32b	64 (36)	시스템 시작 1회	Conv1 weight_loader → 18 PE
conv2_weight_bram	32b	1024 (576)	시스템 시작 1회	Conv2 weight_loader → 192 PE
fc_weight_bram	512b	1024 (720)	시스템 시작 1회	FC streaming read → 16 lane

표 4: 레이어별 독립 weight BMG. Conv1·Conv2는 적재 후 PE 레지스터에 stationary 고정, FC는 spatial마다 streaming read한다. 1 word는 $16ch \times 32\text{-bit SIMD-A}(A = W1 \cdot 2^{17} + W0)$.

데이터-패스 BMG의 폭은 모두 “한 주소에 한 spatial 위치의 전 채널”을 담도록 잡혀, 엔진이 채널을 병렬로 한 번에 read한다 – C1C2 64-bit = 8 IC $\times$ 8b, C2Pool·PoolFC 128-bit = 16 ch $\times$ 8b. 이미지 한 장의 데이터 이동 경로는 다음과 같다: PS → Input BRAM → Conv1 → C1C2 → Conv2 → C2Pool → MaxPool → PoolFC → FC → Output BRAM → PS.

Single-image Dataflow Path

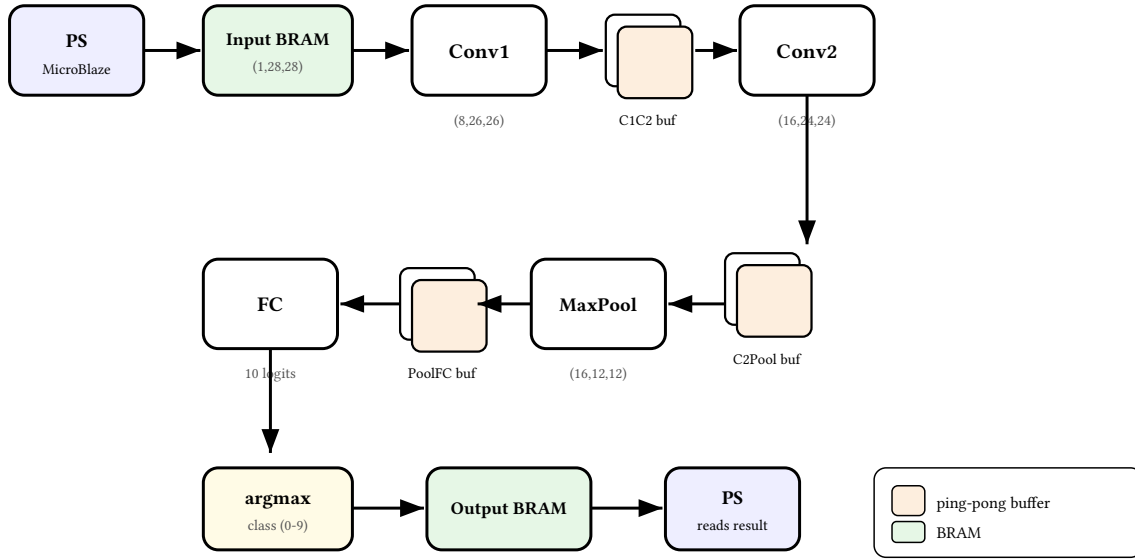


그림 13: 단일 이미지 데이터 이동 경로 (feature-map shape 포함)

3.1.3 Sliding-window 생성 — line buffer / window register 구현

두 컨볼루션 엔진은 Section 2.5 의 line-buffer 스트리밍 원리를 동일한 두 공용 모듈로 구현한다 — `line_buffer(RTL/core/line_buffer.v)`와 `window_register(RTL/core/window_register.v)`.

`line_buffer`는 깊이 DEPTH의 순환 버퍼다. `en=1`인 매 사이클에 현재 포인터 위치를 등록 출력으로 읽고(`dout <= mem[ptr]`) 같은 자리에 새 입력을 덮어쓰기(`mem[ptr] <= din`) 포인터를 순환시킨다. 따라서 한 줄(row)을 통째로 지연시키는 FIFO처럼 동작하며, 등록 출력 1사이클을 향쳐 실행 지연이 DEPTH+1 사이클이 된다. Conv1은 28픽셀 행이라 DEPTH=27 (→ 28사이클 = 1행), Conv2는 26픽셀 행이라 DEPTH=25 (→ 26사이클)로 인스턴스화한다.

```
// 코드 방제 - line_buffer.v: 순환 버퍼 1행 지연 (등록 출력)
always @(posedge clk) begin
    if (rst) begin ptr <= 0; dout <= 0; /* mem 클리어 */ end
    else if (en) begin
        dout <= mem[ptr]; // 현재 위치 읽기(1사이클 지연)
        mem[ptr] <= din; // 새 데이터 쓰기
        ptr <= (ptr == DEPTH-1) ? 0 : ptr + 1; // 순환
    end
end
```

`line_buffer` 2개를 직렬로 두면(BRAM → lb1 → lb2) 세 개의 연속한 행이 동시에 가용해진다. `window_register`는 이 세 행을 받아 각 행마다 3-cell 좌측 시프트 레지스터(`win_r2=처신 행`, `win_r1=lb1`, `win_r0=lb2`)를 굴려, 매 `en` 사이클에 3x3 윈도우 한 개(9타입 k0 k8)를 완성한다. 즉 BRAM에서 한 클럭에 한 픽셀씩 raster-scan으로 흘려보내면, fill 이후 매 사이클 새 윈도우 하나가 나온다. Conv1은 IC가 1개나 이 체인을 1세트, Conv2는 IC가 8개나 8세트(`line_buffer` 16개 + `window_register` 8개)를 병렬로 둔다. Conv1은 출력 채널 round를 바꿀 때(Section 3.2.2) `lb_rst`로 `line buffer`와 `window`를 클리어해 이전 round의 잔류 데이터가 다음 round 윈도우를 오염시키지 않게 한다.

3.1.4 데이터플로우와 분산 FSM 제어

데이터플로우는 **Weight Stationary + Output Stationary, Activation Flowing** 으로 실현된다. 각 PE는 weight를 자기 레지스터에 적재해 추론 동안 고정하고(weight stationary), 부분합(psum)을 누적기에 모으며(output stationary), activation은 BRAM → line buffer → window register → PE로 매 사이클 흐른다(activation flowing).

Section 2.5의 결정대로 stage 사이를 ping-pong BRAM 버퍼로 분리하여, 한 engine이 다음 이미지를 쓰는 동안 다음 engine이 이전 이미지를 읽는다. 중앙 컨트롤러는 없으며, 각 engine이 자체 FSM과 bank-toggle FF를 가지고 stage 간 write_done / read_done (각 1-cycle pulse) 핸드셰이크로 동기화한다.

핸드셰이크는 두 방향의 **signed 3-bit** 차이 카운터를 구현된다. 입력 측 $\text{prior_diff} = (\text{자신의 read 완료 수}) - (\text{이전 stage의 write 완료 수})$ 로, $\text{prior_diff} < 0$ 이면 처리할 이미지가 이전 버퍼에 남아있음을 뜻한다(data_ready). 출력 측은 $\text{after_diff} = (\text{자신의 write 완료 수}) - (\text{다음 stage의 read 완료 수})$ 로, $\text{after_diff} < 2$ 면 출력 bank에 여유가 있음을 뜻한다(output_avail). 두 조건이 모두 참일 때만 다음 이미지 처리를 시작하므로, 어느 stage가 빠르거나 느려도 카운터 자체가 backpressure를 만든다. 카운터는 register이지만 진입 판단은 **next-value**를 조합 논리로 평가하여, 같은 사이클의 pulse가 NBA 1사이클 지연으로 누락되는 race를 막는다(상세는 docs/handshake_counter_nba_race.md).

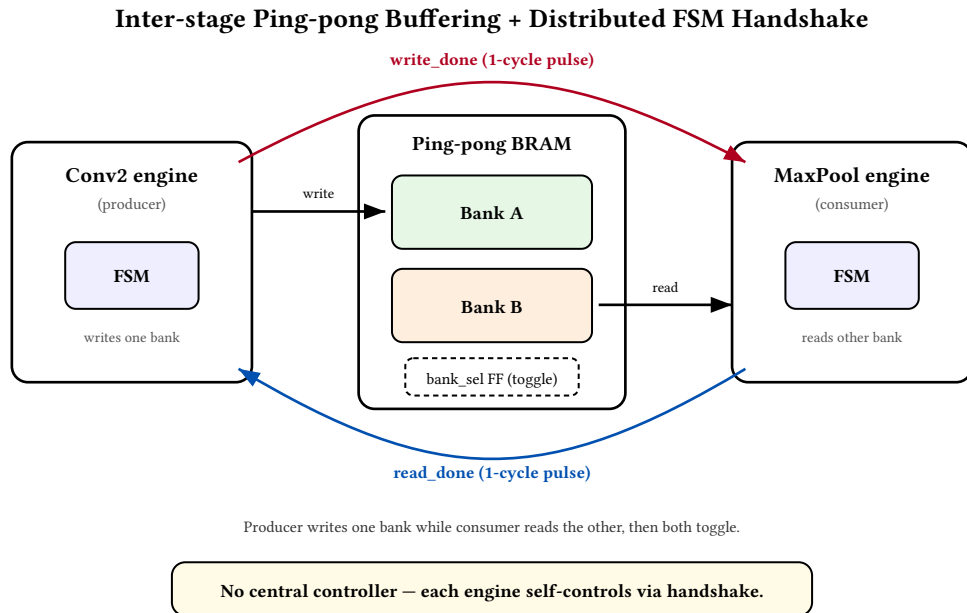


그림 14: stage 간 ping-pong 버퍼 + 분산 FSM 핸드셰이크 (중앙 컨트롤러 없음)

3.2 오버클럭 이전 – Baseline 레이어 엔진

각 레이어 엔진은 “FSM(제어) + 공용/전용 datapath 하위 모듈”로 구성된다. 먼저 모든 엔진이 공유하는 연산 primitive를 보이고, 이어 엔진별 하위 모듈 구성·역할과 정확한 FSM 상태를 전이를 제시한다.

3.2.1 공유 연산 primitive – PE cell과 truncate/ReLU

PE cell (pe_cell.v). 모든 컨볼루션·FC가 공유하는 MAC 단위다. DSP48E1 한 개씩 parameterized weight 레지스터로 구성되며, A 포트(25-bit)에 packed weight, B 포트(18-bit)에 activation을 싣고 48-bit P에서 두 곱 $\text{mul0} = W0 \cdot X$, $\text{mul1} = W1 \cdot X$ 를 분리 추출한다(Section 2.3). 파라미터 DEPTH로 weight 슬롯 수를 정해 레이어별로 재사용한다 – Conv1은 DEPTH=2 (OC round mux), Conv2는 DEPTH=3 (K_col time-mux), FC는 STREAM=1 (weight 레지스터 우회, A 포트 직결). DSP 내부 3단(A/B → M → P) + 모듈 출력 레지스터 1단 = 4-cycle latency다.

```
// 코드 블록 C1 - pe_cell.v: 48-bit P에서 두 곱 분기 + carry 보정 (-128 없음 가정)
wire signed [16:0] p0_raw    = P[16:0];           // W0·X
wire signed [16:0] p1_slot  = {{1{P[32]}}, P[32:17]}; // W1·X 승조
wire signed [16:0] carry_corr = {16'd0, p0_raw[16]}; // [P0<0] carry 보정
wire signed [16:0] p1_raw    = p1_slot + carry_corr;
```

Truncate/ReLU (truncate_relu.v). 모든 레이어 출력 직후의 양자화 단계이다. 24-bit signed 누적값을 $\ggg10$ (산술 우측 시프트)한 뒤 $[-128, 127]$ 로 saturation하고 ReLU를 적용해 8-bit로 출력한다. 음수 분기에서 ReLU와 음수 saturation이 함께 처리된다. 동시 출력 채널 수 N은 Conv1에서 4, Conv2에서 16이다. (양자화 규칙의 명세 정황은 Section 3.2.7.)

```
// 코드 블록 C2 - truncate_relu.v: >>>10 → saturate(±127) → ReLU (음수 분기 통합)
function signed [7:0] sat_relu;
    input signed [13:0] val;           // 24-bit 누적값 >>>10 한 14-bit
    begin
        if      (val > 14'sd127) sat_relu = 8'sd127; // 양수 saturation
        else if (val < 14'sd0)   sat_relu = 8'sd0;   // ReLU + 음수 saturation
        else                    sat_relu = val[7:0]; // 0~127 그대로
    end
endfunction
```

3.2.2 Conv1 엔진

입력 (1, 28, 28) → 출력 (8, 26, 26). Section 2.4.3의 분배대로 **18 DSP** = $K9 \times \text{OCpair2} \times \text{SIMD2}$ 를 쓴다. 9 탭 윈도우를 9 PE로 한 번에 곱하고, SIMD packing으로 한 PE가 두 출력 채널을 내므로 한 round에 4 OC가 나온다. 출력 8채널은 PE를 복제하지 않고 2-round(round0 = OC0-3, round1 = OC4-7)로 나눠 처리하며, round 사이에 line buffer·window를 리셋한다.

하위 모듈	인스턴스	역할
conv1_fsm	1	8-상태 제어 FSM. raster-scan 카운터·flush·round 전환·핸드셰이크·출력 주소 지연
conv1_weight_bram	1	Conv1 SIMD weight BMG (엔진 내부, Port A passthrough)
conv1_weight_loader	1	시작 1회 weight를 read해 18 PE에 1-hot 분배
line_buffer	2	3행 윈도우 생성 (DEPTH=27)
window_register	1	3×3 윈도우 9탭 생성
pe_cell	18	9 PE × 2 group(DEPTH=2). round당 4 OC MAC
conv1_adder_tree	2	9탭 곱 → 24-bit 누적 (9:2 토폴로지, 4-stage)
truncate_relu	1	N=4, 4채널 동시 양자화

표 5: Conv1 엔진 하위 모듈 구성.

데이터패스는 bram_input byte read → line buffer ×2 + window → 18 PE → conv1_adder_tree ×2 → truncate_relu 순이며, 출력 4채널은 c1c2_din의 byte 위치에 실려 같은 주소에 round별 byte-write로 병행된다. FSM은 8상태이며 전이는 다음과 같다.

상태	동작	다음 상태 (조건)
IDLE	대기 (pipe_en=0)	LOAD (data_ready & output_avail)
LOAD	weight 적재 대기	RUN1 (load_done)
RUN1	sel=0, 28×28 스캔 (OC0-3)	FLUSH1 (scan_done)
FLUSH1	파이프라인 drain	LBRST (flush_cnt 뺀)
LBRST	line buffer window 리셋, sel=1	RUN2 (무조건)
RUN2	sel=1, 28×28 재스캔 (OC4-7), 끝에 rdone	FLUSH2 (scan_done)
FLUSH2	파이프라인 drain	DONE (flush_cnt 뺀)
DONE	wdone pulse	IDLE

표 6: Conv1 FSM 상태 전이. RUN1/RUN2 가 두 OC round, FLUSH 가 파이프라인 drain, LBRST 가 round 전환 클리어다.



그림 15: Conv1 FSM 상태 전이도

3.2.3 Conv2 엔진

입력 (8, 26, 26) → 출력 (16, 24, 24). Section 2.4.3 의 분배대로 $192 \text{ DSP} = \text{OCpair}8 \times \text{IC}8 \times \text{Krow}3$ 를 쓴다. 즉 커널 행(K_row 3)은 공간적으로 펼쳐고, 커널 열(K_col 3)은 3사이클에 걸쳐 시분할 누적한다. 8 IC를 한 번에 병렬 처리한다.

하위 모듈	인스턴스	역할
conv2_fsm	1	8-상태 제어 FSM. read 차 표 K_col phase-row wrap-drain-핸드셰이크
conv2_weight_bram	1	Conv2 SIMD weight BMG (엔진 내부)
weight_loader_conv2	1	시작 1회 576 weight를 read해 192 PE에 ID 디코딩 분배
line_buffer	16	IC당 2개(DEPTH=25) - 8 IC 병렬
window_register	8	IC당 1개, 3×3 윈도우
pe_cell	192	DEPTH=3(K_col 승차). $\text{pe_id} = (\text{OC_pair} \cdot 8 + \text{IC}) \cdot 3 + \text{K_row}$
krow_ic_adder_tree	16	$\text{K_row}3 \times \text{IC}8 = 24$ 입력을 22-bit로 합산 (5-stage)
kcol_accumulator	16	K_col 3개를 3사이클에 24-bit로 누적, 마지막에 out_valid
truncate_relu	1	N=16, 16 OC 동시 양자화

표 7: Conv2 엔진 하위 모듈 구성. 누적 경로가 krow_ic_adder_tree(행·IC 합) → kcol_accumulator(열 누적)로 2단계이다.

한 출력 픽셀은 K_col 위상 3개(HOLD/HOLD/ADVANCE)에 걸쳐 완성된다 - COMPUTE_HOLD에서 윈도우를 연산 채 K_col 0-1을, COMPUTE_ADVANCE에서 윈도우를 한 열 진행시키며 K_col 2를 처리하고, 이때 kcol_accumulator가 세 위상의 부분합을 합쳐 한 픽셀을 낸다. 출력 행이 바뀌는 경계에서는 COMPUTE_WRAP(3사이클)이 PE를 멈추지 않고 윈도우만 진행시켜 다음 행 첫 픽셀을 준비한다. 마지막 출력 픽셀 뒤에는 DRAIN이 PE→adder→누적기→BRAM write 파이프라인을 비운다.

상태	동작	다음 상태 (조건)
IDLE	start 대기	LOAD_WEIGHTS (start)
LOAD_WEIGHTS	576 weight 적재 (1회)	DONE (loader_done)
DONE	다음 이미지 대기	PIPELINE_FILL (data_ready & output_avail)
PIPELINE_FILL	line buffer·window 초기 채움	COMPUTE_HOLD (row,col=(2,4))
COMPUTE_HOLD	윈도우 정지, K_col 0/1	COMPUTE_ADVANCE (kw_cnt=1)
COMPUTE_ADVANCE	윈도우 1열 진행, K_col 2	DRAIN(마지막 픽셀) / COMPUTE_WRAP(행 경계) / COMPUTE_HOLD
COMPUTE_WRAP	행 전환, 윈도우 끝 진행 (3사이클)	COMPUTE_HOLD (wrap_cnt=2)
DRAIN	파이프라인 drain	DONE (drain_cnt 만큼)

표 8: Conv2 FSM 상태 전이. HOLD / HOLD / ADVANCE 가 한 픽셀의 K_col 3위상, WRAP 이 출력 행 경계 처리, DRAIN 이 파이프라인 비움이다.

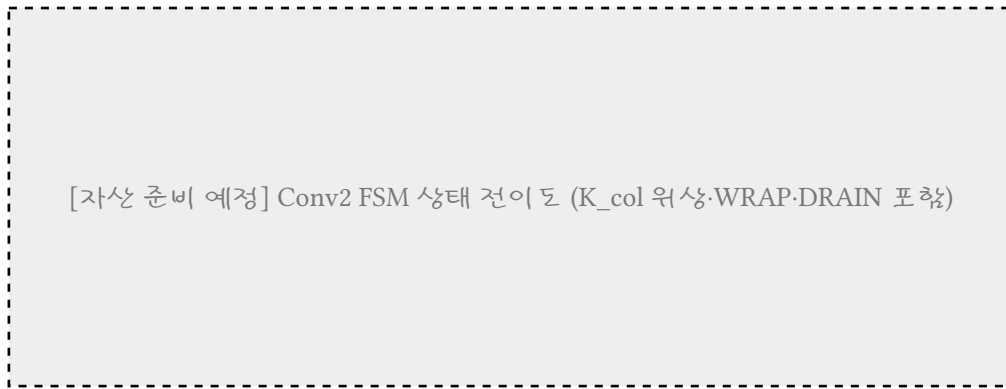


그림 16: Conv2 FSM 상태 전이도 (K_col 위상·WRAP·DRAIN 포함)

3.2.4 MaxPool 엔진

입력 (16, 24, 24) → 출력 (16, 12, 12). 2×2 stride-2 커널을 16채널에 대해 동시에 수행한다.

하위 모듈	인스턴스	역할
maxpool_fsm	1	4-상태 제어 FSM. 7-phase로 2×2 윈도우 4픽셀 read·캡처·출력 주소 생성
max_compare_tree	1	16채널 각각 4-way 커널값을 2-stage 비교로 계산

표 9: MaxPool 엔진 하위 모듈 구성.

FSM은 IDLE → RUN → FLUSH → DONE 의 4상태다. RUN 은 내부 7-phase(0-6)로, 2×2 윈도우의 네 픽셀 주소를 차례로 발행하고 BMG read latency를 고려해 도착분을 p00/p01/p10/p11로 캡처한 뒤 비교를 시작한다. max_compare_tree 는 1단에서 (p00,p01)·(p10,p11)의 행별 커널값을, 2단에서 그 둘의 커널값을 구해 채널당 4-way max를 2사이클에 낸다. 144개 출력 픽셀을 모두 내면 FLUSH로 파이프라인을 비우고 DONE에서 rdone/wdone을 pulse한다.

3.2.5 FC 엔진

입력 2,304(= 16 ch × 144 spatial) → 출력 10 클래스. SIMD packing을 출력 클래스 방향으로 적용해, 한 PE lane이 같은 activation에 대해 두 클래스(even/odd) 기여를 동시에 낸다. 따라서 10 클래스를 5 pair로 묶어 순차 처리한다.

하위 모듈	인스턴스	역할
fc_fsm	1	4-상태 제어 FSM. 5 pair × 144 spatial 스캔, 핸드셰이크
fc_weight_bram	1	FC SIMD-A weight BMG (512b, 엔진 내부)
fc_pe_array	1	16 lane (pe_cell STREAM=1). lane당 p0=W0·X(even), p1=W1·X(odd)
fc_adder_tree	1	16 lane 곱을 even/odd 각각 20-bit로 합산
fc_accumulator	1	pair별 144 spatial 부분합을 logit으로 누적
fc_argmax	1	10 logit의 최대값 인덱스 (4-round 토너먼트)

표 10: FC 엔진 하위 모듈 구성. weight를 레지스터에 적재하지 않고 BMG에서 streaming read해 lane에 직결한다 (spatial마다 weight가 바뀌어 stationary 불가).

weight는 한 word(512-bit)가 16채널 × 32-bit SIMD-A이며, 각 lane이 [ch*32 +: 25]를 pe_cell의 packed weight를 직결받는다. fc_pe_array의 even/odd 출력은 각각 짝수·홀수 클래스 기여이고, fc_adder_tree가 16채널을 합쳐 한 pair의 even/odd 부분합을, fc_accumulator가 144 spatial을 합쳐 두 클래스 logit을 만든다. 5 pair를 모두 돌면 10 logit이 모이고, fc_argmax가 최대값 인덱스를 낸다 — 1-cycle combinational 10-way 비교는 타이밍 위약이므로 10→5→3→2→1의 4-round 토너먼트로 분해해 각 round를 레지스터로 끊었고, strict > 비교로 동률시 낮은 인덱스를 유지한다.

상태	동작	다음 상태 (조건)
IDLE	다음 이미지 대기	COMPUTE (data_ready 또는 start_pulse)
COMPUTE	5 pair × 144 spatial 스캔 (s_first/s_last strobe)	DRAIN (pair4의 s_cnt=143)
DRAIN	파이프라인 drain	DONE (drain_cnt 만료)
DONE	rdone pulse	IDLE

표 11: FC FSM 상태 전이. COMPUTE가 pair·spatial 이중 스캔, DRAIN이 누적기까지의 파이프라인 비움이다. FC는 terminal layer라 출력 측 핸드셰이크가 없다.



그림 17: FC FSM 상태 전이도 (pair·spatial 스캔 + argmax)

3.2.6 PS-PL 인터페이스 (CSR와 main.c)

CSR는 AXI-Lite slave(csr_axi.v)로, PL과의 인터페이스 신호로 enable/start/img_ready(PS→PL)와 img_done/input_consumed(PL→PS)를 보내준다. 타이머는 명세 정의대로 “첫 weight write부터 마지막 output read까지”를 PL에서 카운트하여 PS가 읽을 수 있게 보내준다. (레지스터 주소 맵·비트 필드 표 T2는 csr_axi_slave_lite_v1_0_csr.v와 대조해 부록에서 확정.)

PS 측 펌웨어(main.c)의 흐름은 다음과 같다: weight를 한번 전송한 뒤 타이머를 시작하고, 10,000장 루프에서 다음 이미지를 반대 bank에 preload하면서 img_ready를 주고 done을 polling한다. 입력 bank 여유(input_consumed 기반 backpressure)를 확인하며 진행하고, 모든 이미지가 끝나면 결과를 Output BRAM에서 일괄 read하고 타이머를 멈춰 총 latency를 보고한다. 데이터는 .npy를 C 헤더로 변환해 빌드 시 DRAM에 올리며, 명세 권고대로 UART 전송은 측정 경로에서 배제한다. (코드 방제 C6은 부록.)

3.2.7 양자화 메커니즘의 HW 구현

양자화는 Section 1.5 의 명세 규칙($\gg 10 \rightarrow \text{saturate}(\pm 127) \rightarrow \text{ReLU}$)을 24-bit 누적 $\rightarrow$ 산술 시프트 $\rightarrow$ saturation 경로를 비트-정확하게 재현한다(`truncate_relu`, Section 3.2.1 의 코드 C2). 이는 단순 비트-절단이 아니라 명세 규칙의 정확한 모사이며, 본 과제는 이미 INT8로 양자화된 파라미터의 추론을 정확히 재현하는 문제이지 scale-factor 선행 양자화나 재학습이 아니다(Section 1.5). INT8 정밀도에 맞춘 하드웨어 구성은 SIMD-2 packing PE(Section 2.3), 24-bit accumulator, truncate/round 로직, 비대칭 BRAM 포트·대역폭(Section 3.1.2)이다.

3.3 오버클럭 이후

Baseline의 200 MHz timing closure에 적용한 RTL·제약·구조 변경(reset 분배 트리, high-fanout 제어 net 복제, BMG read latency 정렬 등)은 무엇·결과(WNS)·왜를 한 흐름으로 다루기 위해 Section 5에 모았다. 본 장은 오버클럭 이전의 기능 RTL에 집중한다. 다만 오버클럭 관련 모든 변경은 기능을 바꾸지 않으며(시뮬레이션에서 bit-exact 재검증), “오버클럭 이전/이후” 구분은 정확성과 무관하다.

4 Implementation — Winograd (제안 알고리즘)

Section 2.8 (Complex Winograd 도입)·Section 2.9 (Conv1 2×)의 RTL 구현이다. RTL은 시뮬레이션 bit-exact로 검증되었고, implementation은 171.42 MHz로 timing-met(200 MHz는 미달), 보드 실측은 시간 제약으로 수행하지 못했다.

4.1 Complex F(4×4,3×3) 데이터패스 구현

4.1.1 Winograd 엔진 RTL 구조

Conv2 Winograd 엔진(`conv2_winograd_engine.v`)은 입력 (8,26,26) INT8을 6×6 타일 단위로 받아 (16,24,24) INT8을 내는, 직접 conv2와 동일한 외부 인터페이스의 drop-in 모듈이다. 내부는 네 단계의 데이터패스로 구성된다.

입력 변환(`wino_input_transform`). $V = B^T \cdot d \cdot B$ 를 계산한다. 변환 계수가 $\{0, \pm 1, \pm 4\}$ 뿐이라 곱셈기를 전혀 쓰지 않고 시프트·덧셈으로만 수행한다. 6×6 타일의 8-bit 입력 원소를 변환해 곱셈에 들어갈 14-bit operand들로 펼치며(이 operand들이 Section 2.8 의 46개 곱과 1:1 대응), 3-stage 파이프라인이다.

곱셈 배열(`wino_mul_array` / `wino_dsp_mul`). element-wise 곱을 수행하는 곱셈기 군집으로, 4개 lane(각 IC 그룹) × 46 곱 = 184 DSP 을 쓴다. 각 DSP48E1은 12-bit weight × 14-bit activation $\rightarrow$ 24-bit 곱의 3-stage 구성이다. weight는 per-PE distributed LUTRAM에 저장한다(`wino_weight_loader`가 PS write를 받아 적재). `wino_lane_reduce`가 46개 곱을 Gauss 트릭에 따라 26개 부분항으로 축약하고, `wino_m_assemble`이 결과 대칭으로 36개 M 워치로 확장한다.

출력 변환(`wino_output_transform`). $Y = A^T \cdot M \cdot A$ 를 계산한다. 계수가 $\{0, \pm 1\}$ 뿐이라 역시 곱셈기 없이 덧셈으로만 수행하며, 4-stage 파이프라인으로 28-bit 출력을 낸다.

절단(`wino_truncate`). Winograd 변환의 1/16 스케일과 레이어 양자화 $\gg 10$ 을 결합한 $\gg 14$ 시프트 + saturation + ReLU로 직접 conv2와 동일한 INT8 양자화를 재현한다. (코드 별첨 C7은 부록.)

엔진 latency는 issue로부터 M_valid까지 +11 cycle, tile_out까지 +17 cycle이며(register 단위 분해는 Section 4.1.4), 한 이미지 처리 사이클은 top-module TB 측정 기준 평균 1,348 cyc/img 이다(분해는 Section 4.1.3). (RTL/`conv2_winograd/`)

4.1.2 레이어 내부 제어 — 통신 대신 카운터-구동 FSM

Section 3.1.4 의 분산 FSM은 레이어 사이의 동기 방식이다(`rdone/wdone` 핸드셰이크). 그러나 Winograd 엔진 내부에서는 별개의 선택을 한다. 엔진은 row buffer를 채우는 producer, 타일을 연산하는 consumer, 결과를 C2Pool에 쓰는 writer 세 프로세스가 동시에 되는데, 이 셋은 서로 done 신호를 주고받지 않는다. 대신 각 단의 latency가 데이터에 무관하게 결정적이라는 점을 이용해, 타이밍을 사이클 단위로 미리 계산해 고정 latency 파이프라인으로 만들고 FSM 전이를 전부 내부 카운터 비교로 유도한다. 핸드셰이크

net은 die 전역으로 퍼지는 제어 net이라 높은 클럭에서 route delay 벽이 되는데(Section 5의 reset-broadcast가 같은 부류), 내부 경로는 그 비결정성이 없으므로 미리 계산한 카운터가 더 안전하고 빠르다.

main FSM은 IDLE → LOAD_WEIGHTS → WAIT_IMG → LOAD_INIT → RUN → DRAIN 6상태다. 첫 start에서 LOAD_WEIGHTS로 한 번 진입해 PS가 써둔 pre-transformed weight를 per-PE RAM으로 적재하고(이후 이미지 루프에서는 재진입하지 않으므로 weight 적재는 이미지 latency에 들어가지 않는다), 이후 이미지마다 WAIT_IMG(레이어 간 핸드셰이크 대기) → LOAD_INIT(첫 타일-행 적재) → RUN(36 타일 연산) → DRAIN(마지막 타일 배수)을 돈다. RUN 중 진행을 결정하는 카운터는 `compute_cnt` (0~31, 타일 내 issue 사이클), `tile_cnt` (0~5, 타일-행 내 타일), `trow_cnt` (0~5, 타일-행)이며, 마지막 타일의 마지막 issue를 `trow_cnt==5 ∧ tile_cnt==5 ∧ compute_cnt==31` 라는 순수 카운터 비교로 검출해 DRAIN으로 전이한다. issue 시퀀스는 `compute_cnt`로부터 `oc = compute_cnt[4 : 1]`, `grp = compute_cnt[0]`, weight 주소 = `compute_cnt`로 모두 파생되므로, 한 카운터가 곱셈기 배열·weight read·태그 파이프라인을 동시에 구동한다.

producer 역시 consumer의 카운터를 읽기만 한다. 다음에 적재할 타일-행 `pld_trow`가 consumer가 처리 중인 `trow_cnt` 보다 최대 한 행 앞섰 때까지만 진행하는 부등식($trow_cnt + 1 \geq pld_trow$)이 producer를 앞세우는 유일한 조건이며, consumer→producer 방향 신호는 없다. 이 무신호 동기가 성립하는 근거는 단순한 사이클 부등식이다 – consumer가 한 타일-행을 처리하는 데 $6 \text{ 타일} \times 32 \text{ cyc} = 192 \text{ cyc}$ 가 걸리는 반면, producer가 한 타일-행($6 \text{ row} \times 26 \text{ col}$)을 적재하는 데는 156 read뿐이다. $156 < 192$ 이므로 producer는 항상 먼저 끝나고, consumer가 ping-pong row buffer의 set을 교체할 때 다음 타일-행은 이미 준비되어 있다. 따라서 RUN의 타일-행 경계에서 consumer는 producer를 기다릴 필요가 없으며(무조건 진행), 준비 완료 게이트(`set_ready`)는 오직 최초 LOAD_INIT → RUN 진입에만 쓰인다. 이 36-cycle 여유가 곧 레이어 내부에서 핸드셰이크를 제거할 수 있는 타이밍 마진이다.

두 set의 row buffer를 타일-행 단위로 교대(`set_active="trow_cnt"[0]`)하는 이 ping-pong에는 한 가지 미세 타이밍 보정이 필요하다. C1C2 BRAM은 출력 레지스터를 켜 2-cycle read latency이므로, 타일-행의 마지막 주소를 낸 직후 read enable을 내리면 그 read가 출력까지 전파되지 못해 row buffer의 마지막 원소가 갱신되지 않는다. 이 미갱신은 타일-행 우측 끝 타일의 한 픽셀에서만 오류로 나타나는데, producer FSM에 PDRAIN(3 cycle, read enable 유지) 상태를 두어 마지막 read가 출력 레지스터를 거쳐 row buffer까지 도착한 뒤에야 `set_ready`를 플립으로 써 해소한다. PDRAIN 3 cycle은 BRAM 2-cycle latency에 row buffer write 레지스터 1단을 더한 값이다.

4.1.3 사이클 예산 – 1,348 cyc/img의 구성

한 이미지의 처리 사이클은 위 카운터들의 진행을 그대로 더해 결정된다. steady-state period(연속 이미지에서 `wdone`과 `wdone` 사이 간격)는 네 항의 합이다.

항목	cycle	근거
WAIT_IMG	1	<code>ready_to_compute</code> 판정 1 cycle
LOAD_INIT	160	PIDLE entry 1 + PLOAD 156($6 \text{ row} \times 26 \text{ col}$) + PDRAIN 3
compute (RUN)	1,152	36 타일 $\times$ 32 cyc
마지막 타일 tail (DRAIN)	35	마지막 issue 이후 M·출력·write 배수
steady-state period	1,348	위 네 항의 합 (TB 측정과 일치)

표 12: 표 T11: Winograd 엔진의 이미지당 사이클 분해 – 카운터 진행을 그대로 합산

여기서 **throughput**을 결정하는 것은 **issue rate(1 issue/cyc, FSM 고정)**이지 파이프라인 깊이가 아니다. 한 타일의 마지막 issue 직후 다음 타일의 issue가 곧바로 이어지므로(파이프를 비우지 않으므로) 타일당 실행 사이클은 32로 고정이고, 각 타일의 연산 결과가 흘러나오는 배수(drain)는 다음 타일 연산의 그 값자에 겹쳐 period에 추가되지 않는다. 오직 마지막 타일 – 뒤에 겹칠 타일이 없는 – 의 배수만 RUN 끝에 노출되며, 이것이 표 T11의 tail 35 cycle이다(마지막 issue로부터 `M_valid + 11`, 출력변환·정량·수집 +6, 그리고 16 픽셀 write). 그 결과 오버클럭 라정에서 파이프라인을 깊게 분할해도(입력변환 3-stage, 출력변환 4-

stage 등) 이미지당 사이클은 tail에만 영향을 주 뿐 throughput은 불변이며, 이것이 깊은 파이프라인을 “공짜로” 쓸 수 있는 이유다(Section 5.2).

4.1.4 엔진 타이밍 앵커 – issue에서 출력까지

연산의 모든 사이클은 곱셈기 배열의 **issue** 사이클을 기준으로 정렬된다. issue된 한 데이터가 M_valid에 도달하기까지 거치는 레지스터는 정확히 11단이다 – row buffer read 레지스터 1단, 입력변환 2단(3-stage 중 마지막은 조합), 활성 레지스터 1단, DSP 3단(AREG/BREG → MREG → PREG), lane partial 레지스터 1단, gather 레지스터 2단, M 누적 latch 1단. M_valid 이후로는 출력변환 4단, 절단 1단, 수집 1단을 더 거쳐 issue로부터 tile_out까지 총 17단이다. 한 타일 내에서 OC별 M_valid는 그 OC의 grp1이 issue되는 사이클($c = 2oc + 1$)에 11을 더한 $2oc + 12$ 에 정렬된다.

사건	타일 내 사이클	식
issue (oc, grp)	$c = 0...31$	$grp1(oc) @ c = 2oc + 1$
M_valid(oc)	$2oc + 12$	$grp1\ issue + 11$
출력변환 완료 (Y16)	$2oc + 16$	1. 출력변환 4-stage
절단 출력	$2oc + 17$	1. 절단 1
tile_out 기록	$2oc + 18$	1. 수집 1

표 13: 표 T11b: 한 타일 내 OC별 타이밍 정렬 – oc0은 12/18, oc15는 42/48 사이클에 완성

이 정렬에서 마지막 OC(oc15)의 tile_out 기록(사이클 48)이 곧 타일 완성 신호가 되어 writer를 트리거하고, M의 좌표(oc·타일 위치)는 issue 시점으로부터 같은 단수만큼 지연된 태그 파이프라인(길이 15)으로 추적되어 수집 단에서 올바른 tile_out 위치에 기록된다. 따라서 M 경로의 latency를 한 단이라도 바꾸면 태그 파이프 길이와 수집 인덱스를 함께 옮겨야 하며, 이는 Section 5.2의 carry-bisect 시도가 latency를 +1 늘리며 정렬 묶음을 혼돈 지점과 정확히 일치한다.

4.1.5 Conv1 2× rebalance 구현

Section 2.9 결정대로 Conv1을 DSP 18→36, 2-round→1-round 구조로 바꿨다. PE 군집을 2그룹×9에서 4그룹×9(36 DSP)로 늘려 사이클당 출력을 4 OC에서 8 OC로 두 배로 하고, RUN2·FLUSH2·LBRST 상태를 제거해 FSM을 8상태에서 5상태(IDLE/LOAD/RUN/FLUSH/DONE)로 단순화했다. 이로써 한 이미지가 1,634 cyc에서 **837 cyc**로 줄었다(−797 cyc). 데이터패스 길이와 FLUSH 길이는 불변이다. (RTL/conv1_2x/) 결정·근거는 Section 2.9, 병목 이동 정량 분석은 Section 7.5에 있다.

4.1.6 SIMD packing의 Winograd 적용

Section 2.3의 SIMD packing을 Winograd 곱셈 배열에도 적용한다. 전체 DSP 사용량은 Winograd Conv2(184) + Conv1 2×(36) + FC(16) = 활성 결과 **236/240(98.33%)**로 거의 포화 상태다(Section 6 hierarchical util). weight에 −128이 없으므로 packing이 무손상이고, Winograd 누적에서도 bit-exact가 보장된다(보정의 일관성 측면 가치는 Section 7.1).

4.2 오버클럭 이후

Winograd의 timing closure(200 MHz 시도 → 171.42 MHz met)는 Section 5.2에서 다룬다.

5 Optimization Journey – Timing Closure

기능 완성된 설계를 200 MHz로 닫는 timing closure를 Baseline(Section 5.1, 완료)과 Winograd(Section 5.2, 171.42 MHz met) 두 트랙으로 기술한다. 모든 RTL-제약 변경은 시뮬레이션 bit-exact 재검증으로 기능 불변을 확인했다.

5.1 Baseline Track – 100 → 200 MHz (MET +0.011 ns)

이 설계의 타이밍 벽은 거의 전부 die 전역으로 퍼지는 high-fanout 제어·리셋 net의 route delay였으며, 논리 깊이 문제가 아니었다(위스트 경로의 route 비중이 82~86%). 아래는 시간 순서로 본 closure 과정이다.

5.1.1 왜 오버클럭인가 – compute-bound 판별

가속기의 throughput floor는 conv2 사이클(약 1,799 cyc/img)이 지배한다. 같은 cycle 거동을 더 빠른 클럭에서 돌리면 firmware를 바꾸지 않고도 wall-clock latency가 줄어든다(타이머가 세는 100 MHz cycle 수 자체가 감소). 다만 현재 **latency**가 연산에 묶여 있는가, 데이터 전송에 묶여 있는가를 먼저 확인해야 한다. 이는 예상 연산 시간과 실측을 비교하면 판별된다.

예상 연산 시간 $\approx (\text{conv2 병목 cyc/img}) \times (\text{이미지 수}) \times (\text{클럭 주기})$
 $= 1,799 \times 10,000 \times (1/200\text{MHz}) \approx 90 \text{ ms}$ (compute-only 하한)
 실측(200MHz, feed-overlap 후) = 98 ms

실측이 compute-only 하한에 근접하면 compute-bound이고 클럭 상승이 직접 latency를 환원된다. 크게 벌어지면 여전히 feed bound다. 두 값의 비교와 함의는 Section 6·Section 7.6에서 수치로 다룬다. (목표 클럭은 처음 300 MHz였으나 closure를 진행하며 die 전역 잔여 워밍업이 비현실적으로 커 중간에 200 MHz로 낮췄다 – 맥락은 Section 5.1 후반.)

5.1.2 선행 인프라 – dual-clock CDC

가속기만 빠른 클럭으로 돌리려면 PS-AXI-CSR(100 MHz)와 datapath 사이에 **clock-domain crossing(CDC)**이 필요하다. 두 클럭은 동일 MMCM 출력이라 위상이 정렬되어 비교적 안전하지만, **metastability**를 추가로 줄이기 위해 경계마다 동기화기를 둔다. level 신호는 2-FF 동기화기, 1-cycle pulse 신호는 toggle 기반 동기화기를 쓴다(빠른 클럭에서 펄스가 여러 cycle로 보여 “N배 카운트”되는 것 방지). 사용처는 `cnn_accelerator` 경계 총 5곳이다.

신호	방향	종류	동기화기
start	aclk→clk	pulse	cdc_pulse_sync
img_ready	aclk→clk	pulse	cdc_pulse_sync
img_done	clk→aclk	pulse	cdc_pulse_sync
input_consumed	clk→aclk	pulse	cdc_pulse_sync
enable	aclk→clk	level	cdc_bit_sync

표 14: CDC 경계 5곳 – 펄스 4 + 레벨 1

```
// 코드 블록 C4 – pulse 동기화기: toggle 인코딩 + edge 복원 (cdc_pulse_sync.v)
always @(posedge src_clk) // src: pulse 마다 toggle 발생 (event=edge)
    if (src_rst) tgl <= 1'b0; else if (pulse_in) tgl <= ~tgl;
(* ASYNC_REG="TRUE" *) reg sync0, sync1; reg sync2;
always @(posedge dst_clk) begin // dst: 2-FF 동기 + 1 지연
    sync0 <= tgl; sync1 <= sync0; sync2 <= sync1;
end
assign pulse_out = sync1 ^ sync2; // toggle edge = dst 1-cycle pulse
```

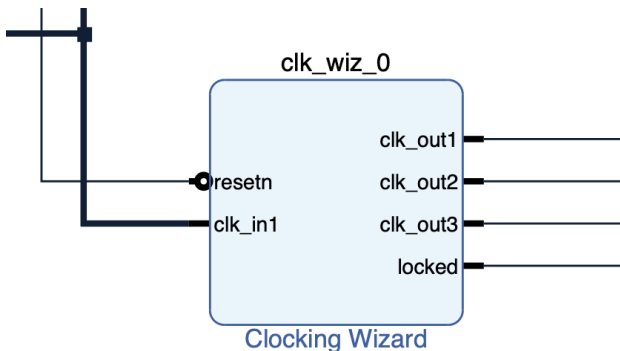


그림 18: 그림 P1: Clocking Wizard 블록

Component Name clk_wiz_0

Board

Clocking Options

Output Clocks

MMCM Settings

Summary

These are the settings based on inputs from previous pages. Any update on this page will override the optimal settings calculated by the wizard

☒ Allow Override Mode

Attribute	Value
BANDWIDTH	OPTIMIZED
CLKFOUT_MULT_F	12.000
CLKFOUT_PHASE	0.000
CLKIN1_PERIOD	10.000
CLKIN2_PERIOD	10.000
COMPENSATION	ZHOLD
DIVCLK_DIVIDE	1
REF_JITTER1	0.010
REF_JITTER2	0.010
STARTUP_WAIT	☐
CLKFOUT_USE_FINE_PS	☐
CLKOUT4_CASCADE	☐

Clk Wizard Port	Renamed Port	MMCM/PLL Port	Divide	Duty Cycle	Phase	Use Fine Ps
clk_out1	clk_out1	CLKOUT0	12.000	0.500	0.000	☐
clk_out2	clk_out2	CLKOUT1	6	0.500	0.000	☐
clk_out3	clk_out3	CLKOUT2	7	0.500	0.000	☐

그림 19: 그림 P2: Clocking Wizard 내부 클럭 설정

5.1.3 MMCM 제약 — “188 MHz는 존재하지 않는다”

Clocking Wizard(MMCM)는 `clk_out1=100` 라 MIG IDELAYCTRL 용 200 MHz가 VCO 주파수를 고정 한다. 따라서 datapath용 `clk_out3` 는 그 VCO의 정수 분주 만 가능하며, 실제로 생성 가능한 값은 {200, 171.4, 166.7, 150, ...}의 이산 집합이다. 함정은 188 MHz를 요청해도 Clocking Wizard가 200 MHz로 스냅한다는 점이다. 이 스냅은 뒤의 silent timing failure의 결정적 빌미가 된다. 따라서 클럭 목표를 정하기 전 `report_clocks` 로 실제 period가 요청값과 같은지 먼저 확인해야 한다.

5.1.4 출발전 — 데이터패스 BRAM에 primitive output register

closure의 첫 조치로, 모든 data-path BRAM(Input / C1C2 / C2Pool / PoolFC)의 출력에 **primitive output register** 를 켜다. 이는 BRAM→로직으로 이어지는 조합 경로를 레지스터로 끊어, BRAM read 직후 다음 파이프라인 경계로 만든다. (BMG 설정 캡처는 docs/ip_spec/.)

5.1.5 초기 -8.6 ns → 100→150 MHz (조합 깊이 + conv2 broadcast)

초기 형성의 WNS는 -8.6 ns 였다. 진단 결과 주범은 두 가지 조합 깊이 병목 — FC argmax의 17-input 9-level 비교 트리와 conv1의 9-input combiner. argmax는 4-round tournament로, conv1 adder는 1→4 stage로 파이프라인화했다.

Timing	Setup Hold Pulse Width
Worst Negative Slack (WNS):	-8.629 ns
Total Negative Slack (TNS):	-22276.062 ns
Number of Failing Endpoints:	17216
Total Number of Endpoints:	137845
Implemented Timing Report	

그림 20: 그림: 초기 WNS -8.6 ns

그 다음 300 MHz 목표 형성에서 WNS -2.99 ns, failing endpoint 110,302개가 전부 **datapath intra-clock** 이었다. 진단 명령은 다음과 같다.

```
report_timing_summary          # 요약 + 워스트 1개
report_timing -setup -max_paths 44 -file paths.rpt # 워스트 전체 덤프 (-file 이 안정적)
report_high_fanout_nets -timing # high-fanout net 식별
```

워스트 경로는 **route 86% / logic 14%** — 배선 거리 문제였다. 원인은 conv2의 제어-weight broadcast(`state / sel / pe_en / packed_w`)가 192개 PE로 fanout되는데, DSP를 226/240(94%)까지 쓰다 보니 PE가 die 전역 DSP 연결에 깔려 broadcast가 die-spanning이 된다는 점이다. DSP 위치는 고정이나 floorplan 불가 → 가능한 방법은 파이프라인 + 드라이버 복제뿐이다. 적용한 조치(누적):

1. `phys_opt -directive max_fanout=32(conv2_fsm state / kw_cnt, weight_loader pe_id) + AggressiveFanoutOpt` → -2.99 → -2.454.
2. weight broadcast +1 register(1회성 load나 compute 무영향).
3. `PE_BC_DELAY` 로 PE 입력단(`sel / pe_en / pe_x`)에 register 복제 → broadcast가 PE 클러스터 근처 replica에서 출발 → -2.187.
4. weight_loader 주소 `pe_id`를 6-level 중첩 곱셈에서 단조증가 accumulator로(조합 깊이 6→1).

Implementation	Summary Route Status Failed Nets
Status: Messages	✓ write Johnson-Combs
Messages: 112 warnings	112 warnings
Active exit: 0	0
Run: mcs8000g204-1	mcs8000g204-1
Settings: Basic Implementation Defaults	Basic Implementation Defaults
Report Strategy: Report	Report
Implementation: Report	Report
Timing	Setup Hold Pulse Width
Worst Negative Slack (WNS):	-2.187 ns
Total Negative Slack (TNS):	-54732.820 ns
Number of Failing Endpoints:	57439
Total Number of Endpoints:	129841
Implemented Timing Report	

그림 21: 그림: -2.99 ns (conv2 broadcast)

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): -2.187 ns	Worst Hold Slack (WHS): 0.052 ns	Worst Pulse Width Slack (WPWS): 0.206 ns	
Total Negative Slack (TNS): -54732.820 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 57439	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 129841	Total Number of Endpoints: 129799	Total Number of Endpoints: 56941	
Timing constraints are not met.			

그림 22: 그림: -2.187 ns (PE_BC_DELAY 후)

300 MHz는 broadcast를 달아도 reset-FSM 잔여 워킹(-1.7~-1.94)이 die 전역에 남아 비현실적이었다. 따라서 목표를 낮춰 150 MHz에서 깨끗이 달았고, 150 MHz 핵심 빈드로 MNIST 10,000장을 보드에서 10,000/10,000 분류함을 확정 하였다.

5.1.6 150→200 MHz – silent timing failure 진단, 그리고 reset fanout

문제 발생. 150 이후 “200 MHz 빈드인데 wall-clock이 100 MHz와 동일(18.77 M cyc)”이라는 측정이 나왔다. 처음에는 이를 “오버클럭을 더 해도 안 죽어든다 = feed bound”로 해석할 뻔했다.

원인 파악. 그러나 그 “200 MHz 빈드”는 사실 **silently fail**한 빈드 였다 – 188 MHz로 설정 → Clocking Wizard가 200 MHz로 스냅 → 실제로는 200 MHz로 돌면서 reset 경호(-1.94 ns)가 워킹 → 분산 FSM-in-flight 카운터 구조가 desync되어 중간에 멈춤. Vivado는 요청 클럭(188) 기준으로 통과시켰으나 실제 클럭(200)에서는 워킹이었던 것이다. 즉 그 latency 비교는 깨진 빈드끼리의 비교였으므로 폐기했다. 교훈은 셋이다: ① report_clocks로 실제 period ≠ 요청 period인지 확인(스냅 탐지), ② **slow(signoff) corner**의 양수 WNS와 신뢰, ③ 타이밍이 깨진 HW 측정은 성능 근거로 쓸 수 없다.

Implementation	Summary	Route Status	Failed Nets
Status: Complete Messages: 210 warnings Active run: impl_1 Part: xc7a100tbg324-1 Strategy: Vivado Implementation Defaults Report Strategy: Vivado Implementation Default Reports Incremental Implementation: None			
Timing	Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.04 ns Total Negative Slack (TNS): 0 ns Number of Failing Endpoints: 0 Total Number of Endpoints: 181418 Implemented Timing Report			
Power	Summary	On-Chip	
Total On-Chip Power: 2.508 W Junction Temperature: 36.4 °C Thermal Margin: 48.6 °C (10.5 W) Effective BIA: 4.6 °C/W Power supplied to off-chip device: 0 W Confidence level: low Implemented Power Report			

그림 23: 그림: silent-fail 의심 빈드 (요청 클럭 기준 통과, 실제 클럭에서 워킹)

해결. 이 깨달음이 방향을 정했다 – “reset 경호(-1.94)부터 달자”. 남은 최대 워킹이 바로 달일 reset net이었다: rst_sync → BUFG → (fanout 41,323) → DSP/register, -1.94 ns, route 85%, die 전역. 달일 net이 datapath 전 레지스터(약 41k)로 직접 fanout되어 BUFG 글로벌 라우팅으로 die 끝까지 가는 데 너무 오래 걸렸다. reset 부하를 없애는 tie-0 방식은 기능 거동을 바꾸고 fragile하여 기각, **reset** 복제 트리 를 채택했다(분배 구조만 바꾸므로 기능 완전 불변).

```
// 코드 블록 C5 – cnn_accelerator.v: async-assert / sync-deassert 복제 트리
(* max_fanout = 32 *) reg rst_l1;      // trunk (few copies)
always @(posedge clk or negedge resetn)
    if (!resetn) rst_l1 <= 1'b1; else rst_l1 <= rst_sync;
(* max_fanout = 128 *) reg rst_leaf;  // leaf (datapath 근처로 대량 복제)
always @(posedge clk or negedge resetn)
    if (!resetn) rst_leaf <= 1'b1; else rst_leaf <= rst_l1;
wire rst = rst_leaf;
```

max_fanout이 핵심기에게 rst_sync(1) → rst_l1(~11) → rst_leaf(~323) → datapath(~41k) 트리를 자동 생성시키고, 각 leaf 복제본을 자기 클러스터 근처에 배치하게 하여 high-fanout net을 짧은 local net 다수로 쪼갬다. async-assert이므로 모든 단이 reset을 즉시(스큐 0) assert하고, deassert와 +2 clk 균일 지연된다(전체 idle-start나 무해). 이후 잔여 워킹을 단계적으로 달았다.

단계	WNS(ns)	Fail	위스트 경향	조치
reset 트리(초기)	-0.154	44	pe_id_reg → pe_load_en_dec_r (route 82%)	reset -1.94 소멸
1. phys_opt(default)	-0.102	31	fsm/state → lb2/mem/CE (route 86%)	default plateau
1. shift_en mf=16	-0.098	1	(lb2 거의 달림)	RTL 복제(캡처 없음)
1. phys_opt Aggr.Explore	+0.011	0	—	MET

표 15: 표 T4: 200 MHz WNS 타임스톤

세 번째 행의 새 위스트는 conv2 shift_en(line buffer clock-enable)이었다. state.kw_cnt 는 이미 max_fanout=32 였으나 shift_en에 빠져 있어 한 줄로 복제 속성을 부여했다.

```
(* max_fanout = 16 *) wire fsm_shift_en; // conv2_engine.v - zero-latency, 기능 불변
```

마지막 한 곳은 phys_opt directive였다. default phys_opt_design 은 -0.154→-0.102에서 plateau였고, phys_opt_design -directive AggressiveExplore 가 -0.098 → **+0.011(0 failing)** 로 마감했다.

⚠ 재현성 함정(필수 기록): 위 AggressiveExplore는 interactive phys_opt의 in-memory 결과다. impl을 재 실행하면 -0.098로 되 돌아간다. 재현하려면 impl strategy에 post-route phys_opt(AggressiveExplore) 를 명시적으로 넣어야 한다.

최종 결과는 **200 MHz timing CLOSED – WNS +0.011, TNS 0.000, WHS +0.002** 다. 이는 slow(signoff) corner의 양수 WNS이므로 앞의 -1.94 silent fail과 근본적으로 다른 정식 충족이다.

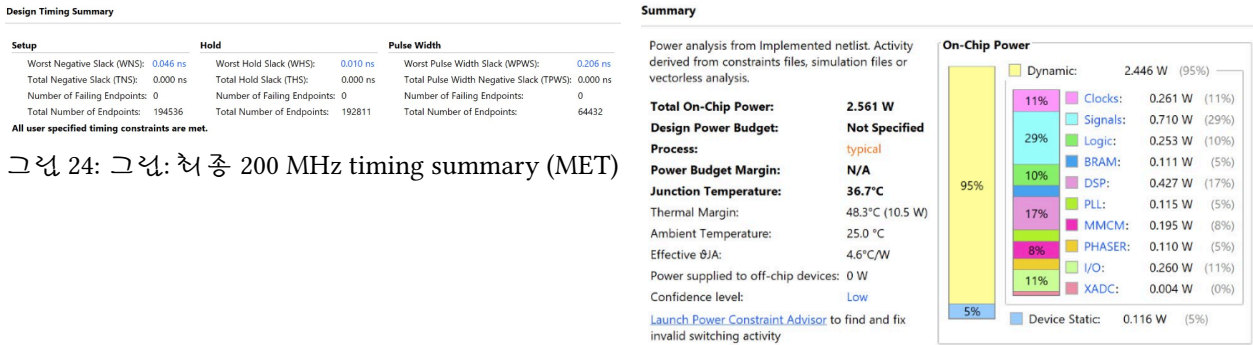


그림 25: 그림: 200 MHz power summary (해석은 Section 7.4)

이 설계의 datapath 타이밍 벽은 대부분 high-fanout 제어-reset net의 route delay였고(로직 깊이가 아님), 흔 라적인 방법은 max_fanout 으로 드라이버를 클러스터 근처에 복제하는 것이었다. 위스트 하나를 달으면 다른 위스트가 드러나는 라정이 반복되었다(상세는 Section 7).

5.2 Winograd Track – 200 MHz 시 도 끝에 171.42 MHz로 met

Winograd 엔진도 같은 closure 방법을 적용했으나, 곱셈을 줄이는 대신 transform network-gather 구조가 추가되어 route congestion이라는 새로운 변수가 생겼다. RTL은 시뮬레이션 bit-exact로 검증됐고 해상도 fit했지만, 목표였던 200 MHz는 끝내 달지 못하고 171.42 MHz에서 timing-met 으로 마무리했다.

해상도 단계 – LUT overflow. Winograd weight는 offline에서 미리 변환된 상수($U = GgG^T$)이므로, 첫 설계는 이 변환 상수를 conv2_winograd 내부 상수 ROM에 baked하여 모듈이 자체 weight를 들고 있도록 했다(외부 weight BRAM-loader 불필요). 그러나 이 상수 ROM이 LUT로 해상되어 LUT를 78.7K (63.4K 초과)까지 차지해 배치가 실패했다. 두 가지로 해결했다: ReLU 출력 범위 분석으로 데이터 비트폭을 줄이고(VW 16→14, MW 32→25, YW 36→28 등), baked ROM을 PS-writable BMG(wino_weight_bram)로 바꿔 같은 상수를 BRAM에서 읽도록(ROM과 bit-identical) 옮겼다. 결과 LUT 75.88%로 fit.

라우팅 단계 - 병목 제거. routing에서 worst path를 하나씩 제거했다. baseline과 달리 병목의 성격은 route congestion이었다.

- **-0.04 ns**: row buffer write broadcast(fanout 312, route 93%) → per-PE distributed LUTRAM 전환.
- **-1.74 ns**: output transform의 20-level 조합 깊이 → OT를 1-cycle에서 4-stage로 분할.
- **-0.41 ns**: 행성기가 4-lane weight 레지스터를 equiv-merge해 fanout 2,944 폭증 → (* keep, max_fanout *)로 lane별 복제 보존.
- **-0.34 ns**: 잔여 40 endpoint를 IT·tile6·wm·gather 4개 class로 증합.
- **-0.094 ns, 7 EP**: 세 class(M/B/D)의 낮은 M은 25-bit 연속 누적 carry chain, B는 tile6_q (2,304-bit) die-spanning scatter, D는 동일 25-bit 가산.

-0.094 ns / 7 EP의 두 부류. 낮은 7 endpoint는 logic-bound와 route-bound로 나뉜다. worst path는 reduce 출력 gpim_q에서 m_im_flat까지의 경로로, 한 cycle 안에 25-bit 가산과 conj 부호반전 carry chain이 직렬로 놓여 있다.

```
Slack (VIOLATED): -0.094 ns
Source: conv2/gpim_q_reg[25][4]/C
Destination: conv2/m_im_flat_reg[874]/D
Path Group: clk_out3 (200 MHz target)
Data Path Delay: 4.942 ns (logic 2.851 ns = 57.7%, route 2.091 ns = 42.3%)
```

logic이 57.7%로 지배적이므로 이 경로는 조합 깊이가 한계인 logic-bound다. 반면 B (tile6_q→u_it, 2,304-bit die-spanning scatter)와 D는 logic이 얇고 route가 68~85%를 차지하는 route-bound다. 두 부류는 처방이 다르다 - logic-bound는 RTL 파이프라인 분할로 조합 깊이를 줄여야 달리고, route-bound는 배치(over-constrain max_fanout 복제)로 배선을 줄여야 달린다. 한쪽 처방을 다른 부류에 적용하면 혼란이 없거나 역혼란이 난다.

route-bound 처방 - over-constraining. route 벽을 줄이는 방법이 over-constraining이다. place/route 동안 set_clock_uncertainty -setup 0.7로 크락을 인위적으로 더 조여 placer-router가 critical net을 더 짧게 폭도록 강제하고, signoff 직전 -setup 0으로 되돌려 실제 제약 기준 WNS를 측정한다. impl strategy 혹은 (place PRE / post-route phys-opt PRE)으로 자동화해 Design Runs의 WNS 역이 곧 signoff 값이 되게 했다. 핵심은 over-constrain이 배치 자유도만 더 쓰게 할 뿐 조합 깊이는 못 줄이므로 정확히 **route-bound** 벽에만 듣는다는 점이다 - logic 벽 M에는 무효다.

그 순환을 통제 실험으로 분리했다 - 동일 행성에서 directive와 over-constrain을 달리한 세 impl run을 비교했다.

impl run	directive	over-constrain	WNS @171.42
stock (tool 기본)	Default	—	+0.068
strong directives	ExtraTimingOpt + phys-opt	—	+0.026
over-constrained	ExtraTimingOpt + phys-opt	+0.7 ns	+0.180

표 16: 표 T13: over-constrain의 순환 - 동일 synth.directive에서 over-constrain만 추가. directive 강화만으론 비달조(+0.026 < +0.068, placement은 확률적)이고, over-constrain이 약 +0.11 ns의 실험 마진을 만든다 (route-bound 벽이 지배하는 171.42 MHz)

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	F...	Methodology	LUT	FF	BRAM	URAM	DSP	Start	Elapsed	Run Strategy
✓ synth_1 (active)	constrs_1	synth_design Complete										0	0	0	0	0	6/19/26	00:04:01	Vivado Synthesis Defaults (Vivado Syn
✓ impl_1	constrs_1	Implementation Out-of-date	-0.099	-0.375	0.010	0.000	8.246	0.000	2.989	0	17 Warn	42397	64920	56.5	0	236	6/19/26	00:12:55	Vivado Implementation Defaults* (Vivad
✓ impl_oc700_signoff (active)	constrs_1	phys_opt_design (Post-Route) Complete	0.190	0.000	0.020	0.000	8.696	0.000	2.949	0	22 Warn	39727	59958	56.5	0	236	6/19/26	00:21:48	Vivado Implementation Defaults* (Vivad
✓ impl_stock_ref	constrs_1	route_design Complete	0.068	0.000	0.007	0.000	8.984	0.000	2.898	0	21 Warn	39515	59761	56.5	0	236	6/19/26	00:14:16	Vivado Implementation Defaults* (Vivad
✓ impl_extraTO_noDC	constrs_1	phys_opt_design (Post-Route) Complete	0.026	0.000	0.051	0.000	8.889	0.000	2.953	0	21 Warn	39585	59894	56.5	0	236	6/19/26	00:19:45	Vivado Implementation Defaults* (Vivad
Out-of-Context Module Runs																			
✓ ccm_accelerator_system		Submodule Runs Complete																	Win

그림 26: 그림: 세 impl run의 Design Runs 비교 - 동일 synth에서 레시피를 달리해 WNS-util-power 대조

이 따진으로 171.42 MHz를 달았다. 폰 T13은 통제 비교용 run이고, 제측 signoff build는 동일 over-constrain으로 WNS +0.222로 달렸다(그림 F11). 200 MHz에서는 같은 over-constrain으로 route 벽 B:D는 죽었으나 logic 벽 M의 조항 깊이가 그대로 남아 closure에 이르지 못했다.

한계에 부딪힌 — **bisect**와 **floorplan** 둘 다 실패. -0.094를 달기 위한 두 시도가 모두 역효과였다. (i) **M carry-bisect(13+12 분할)**: M은 달렸으나(-0.094→0) +1 latency와 새 레지스터 배치 churn으로 **B class**가 -0.088→-0.150 히커 하고 Fmax가 196→187 MHz로 떨어져 revert. (ii) **Floorplan(pblock 양측)**: gather→central-reduce 구조가 lane 입력 4-way fan-in과 극로벌 출력 fan-out을 동시에 갖는 net-bound 구조라, 양측하면 한쪽 net이 반드시 늘어났다 → 강제 양측 시 WNS -1.116으로 10배 약화 → 본질적으로 floorplan 부적합.

현재 위치 — **171.42 MHz MET**. 200 MHz는 한 worst path를 죽이면 다른 worst path가 드러나는 과정이 반복되어 달지 못했고, 171.42 MHz로 **fallback** 했다. 171.42 MHz에서는 **WNS +0.222 ns, TNS 0.000, 0 failing endpoint**로 **timing**이 달렸다 (period 5.833 ns, 그림 F11). 200 MHz만 미안이다. 200 MHz closure에 남은 처방은 두 부류에 각각 대응한다. route 벽 B:D는 over-constrain으로 죽어들고(폰 T13), logic 벽 M은 worst path의 carry chain을 register로 분할해 조항 깊이를 낮춘다 — msum 가상기와 wino_m_assemble (부호반전) 사이에 register 1단을 삽입한 형태다.

```
// msum = acc + gpre_q (가산) 과 wino_m_assemble (conj 부호반전) 사이에 register 1단 삽입
reg signed [MW-1:0] msum_im_q [0:25];
always @(posedge clk) if (mul_en_q4)
  for (kk = 0; kk < 26; kk = kk + 1)
    msum_im_q[kk] <= acc_im[kk] + gpim_q[kk]; // carry chain #1 (가산) 만 한 cycle
wino_m_assemble u_asm (.sim_flat(msum_q_im_flat), ...); // 부호반전은 다음 cycle 로 분리
```

이 분할은 latency를 1 cycle 늘리되 throughput(cyc/img)은 불변이며, conv2 standalone과 full pipeline TB 모두 100/100 bit-exact로 동작 보존을 확인하였다. 앞선 carry-bisect는 M을 달았으나 새 레지스터가 route 벽 B:D의 배치를 churn시켜 Fmax가 196에서 187 MHz로 낮아졌으므로, 이 분할은 over-constrain과 함께 적용해 route 벽의 churn을 억제한다.

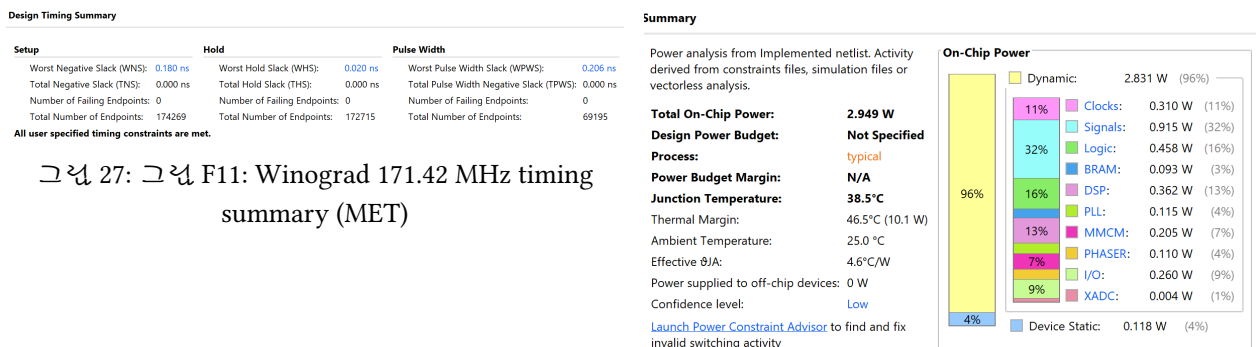


그림 27: 그림 F11: Winograd 171.42 MHz timing summary (MET)

그림 28: 그림: Winograd 171.42 MHz power summary

대량 보드 실측은 수행하지 못했다 — 보드 제측 마감 시점에 200 MHz fallback이 확정되지 않아 최종 Winograd 비트스트림의 HW bring-up-latency 측정을 하지 못했다. 따라서 Winograd의 1만 장 latency는 시뮬레이션 bit-exact + implementation timing(171.42 MHz met) + TB 측정 cycle로부터 추정한다: compute-bound 가정 시 약 $1,341 \times 10, \frac{0.000}{171.42}$ MHz ≈ 78 ms(feed 오버랩 무시), 200 MHz가 달렸다면 ~67 ms였을 것이다. 모두 보드 실측이 아닌 추정임을 명시한다.

5.3 두 트랙 종합

Baseline은 200 MHz MET(WNS +0.011 ns)이고 보드 실측까지 완료 (~98 ms)했고, Winograd는 200 MHz를 달지 못해 171.42 MHz MET(WNS +0.222 ns)으로 fallback했으며 보드 실측은 시간상 못 해 추정(~78 ms)에 그친다.

항목	Baseline (Direct)	Winograd	
클럭	200 MHz	171.42 MHz	WNS
+0.011 ns (MET)	+0.222 ns (MET)	검증	보드 실행
시뮬 bit-exact only	정확도	10,000/10,000 (실행)	100/100 (시뮬, 보드 측정)
cyc/img	~1,799 (conv2 floor)	~1,341 (TB)	1'당 latency
~98 ms (실행)	~78 ms (추정)		

표 17: 표 T12: Baseline vs Winograd closure 종합 (실행/추정 구분)

같은 closure 방법(max_fanout 복제, 파이프라인 분할, phys_opt directive)이 성격이 다른 두 설계 모두에 적용되었다. 다만 Winograd는 gather 구조의 route 특성 때문에 마지막 구간을 달기가 더 어려워, 같은 200 MHz 목표에 도달하지 못하고 한 단계 낮은 이산 클럭에서 달렸다.

6 Results

6.1 Baseline — 오버클럭 이전: 기능 검증

기능 검증의 기준은 PyTorch reference 모델(golden)이다. golden은 코부터 레이어별 입력·기대출력을 hex 코 생성하고, 각 레이어의 출력이 INT8 단위 코 golden과 정확히 일치하는지(bit-exact)를 모듈별 테스트벤치마크 전체 통합 테스트벤치로 확인했다.

[자산 준비 예정] 그림 F12: golden → hex → RTL bit-exact 검증 파이프라인

그림 29: 그림 F12: golden → hex → RTL bit-exact 검증 파이프라인

6.1.1 PE cell — 2^{24} exhaustive

PE cell은 weight·activation 입력 조합 전체($2^{24} \approx 16.7\text{M}$)에 대한 exhaustive 검증으로 SIMD packing이 모든 INT8 경우에 bit-exact ($W_1 = -128$ corner 포함)임을 증명했다. 이는 Section 2.3 packing 주장의 직접 근거다.

[자산 준비 예정] 그림 W1: pe_cell 2^{24} exhaustive PASS 로그 (캡처 예정)

그림 30: 그림 W1: pe_cell 2^{24} exhaustive PASS 로그 (캡처 예정)

6.1.2 Conv1 / Conv2 — 단위 TB (로그 + 파형)

Conv1·Conv2 각각의 단위 테스트벤치로 레이어 출력이 golden과 bit-exact임을 확인했다. 파형으로는 line buffer → window register로 3×3 윈도우가 매 사이클 생성되는 과정과 PE 누적 타이밍을 분석한다.

[자산 준비 예정] 그림 W2: conv1 PASS 로그 + 윈도우/누적 파형 / 그림 W3: conv2 PASS + K_col 누적 (캡처 예정)

그림 31: 그림 W2: conv1 PASS 로그 + 윈도우/누적 파형 / 그림 W3: conv2 PASS + K_col 누적 (캡처 예정)

6.1.3 FC / argmax – 단위 TB (로그 + 파형)

FC는 2,304→10 누적이 golden과 일치함을, argmax는 4-round tournament가 최댓값 인덱스를 올바르게(낮은 인덱스 우선 tie-break) 내는지 확인한다. 파형으로 logit 누적 완료 → argmax 4-cycle latency를 분석한다.

[자산 준비 예정] 그림 W4: fc PASS + logit 누적 / 그림 W5: argmax PASS + tournament (캡처 예정)

그림 32: 그림 W4: fc PASS + logit 누적 / 그림 W5: argmax PASS + tournament (캡처 예정)

6.1.4 MaxPool – 단위 TB (로그)

MaxPool은 동작이 단순(2×2 조항 비교)하므로 PASS 로그만 실행한다.

[자산 준비 예정] 그림 W6: maxpool PASS 로그 (캡처 예정)

그림 33: 그림 W6: maxpool PASS 로그 (캡처 예정)

6.1.5 통행 TB & ping-pong

전체 파이프라인 통행 테스트 벤치로 MNIST 이미지에 대해 logit이 bit-exact임을 확인하고, ping-pong bank toggle과 stage 간 핸드셰이크 타이밍을 파형으로 분석한다.

[자산 준비 예정] 그림 W7: 통행 TB PASS 로그 + ping-pong/handshake 파형 (캡처 예정)

그림 34: 그림 W7: 통행 TB PASS 로그 + ping-pong/handshake 파형 (캡처 예정)

6.2 Baseline – 오버클럭 이후: 보드 구현 & 실행

Timing. 200 MHz에서 timing closure를 완료했다(WNS +0.011 ns @ slow corner, MET). 단계별 서사상 WNS 추적 표(표 T4)는 Section 5.1에 있으며, 여기서는 최종 signoff 값만 인용한다.

Resource utilization. baseline은 DSP 226/240(94%) (Conv2 192 + Conv1 18 + FC 16)을 쓴다. LUT / FF / BRAM을 포함한 전체 사용량은 아래 implementation 리포트와 같다.

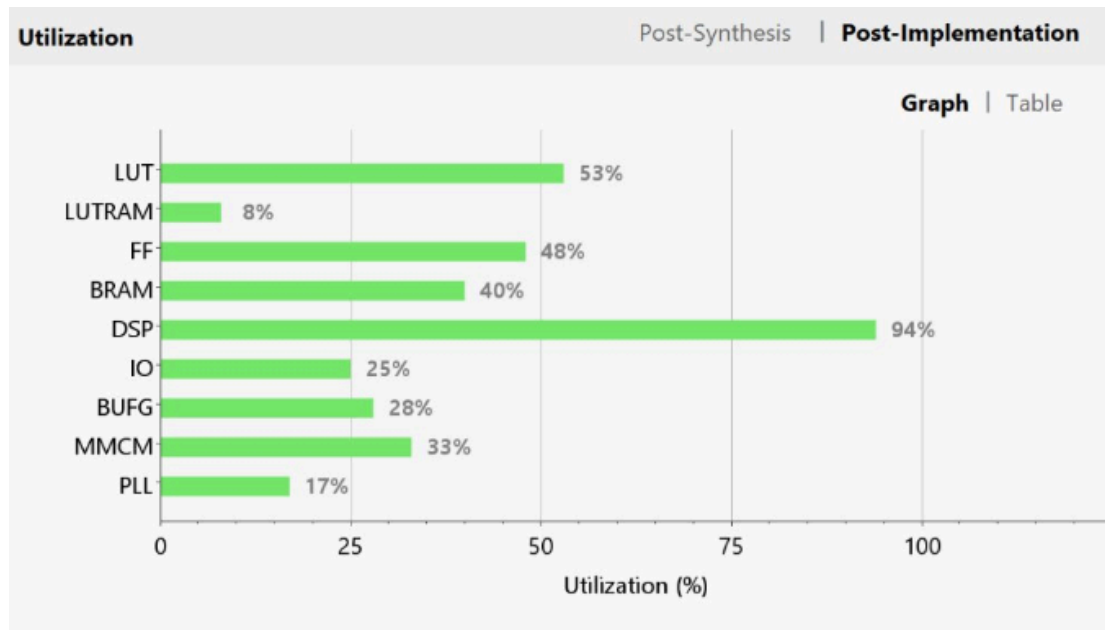


그림 35: 그림: Baseline 200 MHz utilization 리포트

Power (명세 우선순위 #2). default setup으로 측정할 power summary는 아래와 같다. 명세 p.7의 요구대로 power 리포트 setup을 변경하지 않았으며, 트레이드오프 해석은 Section 7.4에 둔다.

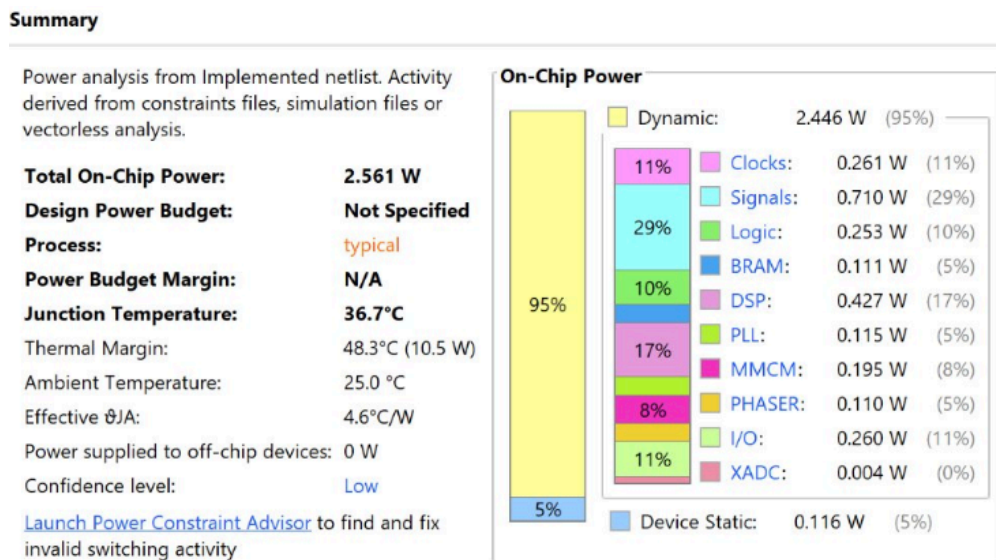


그림 36: 그림: Baseline 200 MHz power summary (default setup)

보드 실행 결과. clean 빌드(WNS +0.011)에서 측정했으므로 수치 자체가 신뢰 가능하다.

- 정확도(SW 대비): MNIST 10,000장에 대해 **10,000 / 10,000** 일치 (100%).
- **Latency:** 최초 200 MHz 실행 **108.9 ms** (10,896,290 cycle @ 100 MHz 타이머). 이 108.9 ms는 100 MHz baseline(0.188 s) 대비 1.72×, 150 MHz(0.128 s) 대비 1.17×이다. 이후 Vitis feed-overlap 최적화로 약 **98 ms** 까지 더 단축했다(0.188 s 대비 1.92×). 즉 클럭만으로는 1.72×, feed 최적화를 더하면 1.92×다.
- 프로파일: 실행 로그상 in-CDMA(blocking) 입력 feed가 전체의 72%(7,905,500 cycle, 100 MHz 도메인)를 차지한다. 가속기 클럭을 2배로 올려도 compute slice만 양축되므로 1.72×에 그쳤다 — 즉 다음 병목이 연산이 아니라 **PS-PL feed** 인을 데이터가 가리킨다(해석은 Section 7.6).

단계	1'각각 latency	baseline 대비
100 MHz baseline	0.188 s	1.00×
150 MHz	0.128 s	1.47×
200 MHz	108.9 ms	1.72×
200 MHz + feed-overlap	~98 ms	1.92×

표 18: 표 T6: end-to-end latency 진행 (검증된 지점)

```

=== CNN Accelerator Test (phase-2, N=10000) ===
[1] weights (direct): conv1=36 conv2=576 fc=11520
[2] enable + start
[3] 10000 images (can_load-paced, inter-image pipelined)...
... 1/10000
... 1025/10000
... 2049/10000
... 3073/10000
... 4097/10000
... 5121/10000
... 6145/10000
... 7169/10000
... 8193/10000
... 9217/10000
img 0: result=5 exp=5 OK
img 1: result=0 exp=0 OK
img 2: result=4 exp=4 OK
img 3: result=1 exp=1 OK
img 4: result=9 exp=9 OK
img 5: result=2 exp=2 OK
img 6: result=1 exp=1 OK
img 7: result=3 exp=3 OK

=== Result ===
class match : 10000 / 10000
latency      : 9799994 cyc ~97999 us wall-clock [100MHz timer]
PS time      : in-CDMA(blocking) 56% (5489027 cyc)  waiting-can_load 31% (3048617 cyc)

```

그림 37: 그림: 200 MHz + feed-overlap 보드 실행 (10000/10000, 98 ms)

6.3 Winograd — 오버클럭 이전: bit-exact 검증

Winograd 데이터패스는 시뮬레이션 정수 검증을 통과했다. Winograd를 적용한 **top-module TB**는 logit bit-exact + bram_output readback 모두 **100/100 PASS** (FINAL: results 100/100, throughput 132,759 cyc total, avg 1,341 cyc/img)이다. Conv1 2× rebalance gate는 별도로 40/40 bit-exact이며, 두 결과는 서로 다른 테스트벤치이므로 혼동 없이 분리해 표기한다.


```

Tcl Console x Messages Log
[TB] img 82 : logit PASS @cyc 125612
[TB] img 83 : logit PASS @cyc 126953
[TB] img 84 : logit PASS @cyc 128294
[TB] img 85 : logit PASS @cyc 129635
[TB] img 86 : logit PASS @cyc 130976
[TB] img 87 : logit PASS @cyc 132317
[TB] img 88 : logit PASS @cyc 133658
[TB] img 89 : logit PASS @cyc 134999
[TB] img 90 : logit PASS @cyc 136340
[TB] img 91 : logit PASS @cyc 137681
[TB] img 92 : logit PASS @cyc 139022
[TB] img 93 : logit PASS @cyc 140363
[TB] img 94 : logit PASS @cyc 141704
[TB] img 95 : logit PASS @cyc 143045
[TB] img 96 : logit PASS @cyc 144386
[TB] img 97 : logit PASS @cyc 145727
[TB] img 98 : logit PASS @cyc 147068
[TB] img 99 : logit PASS @cyc 148409

=====
FINAL : results 100 / 100
images PASS : 100 / 100
throughput : 132759 cyc total (img0..99), avg 1341 cyc/img
bram_output readback : 100 / 100
*** PASS *** (logit bit-exact + bram_output readback, overlap)
=====
$finish called at time : 1484690 ns : File "C:/Users/gimdohyeon/CNN_
run: Time (s): cpu = 00:02:56 ; elapsed = 00:08:50 . Memory (MB): p

```

그림 38: 그림 F10: Winograd top-module TB 결과 (100/100, avg 1,341 cyc/img)

검증 기준은 정수 golden이다 – 하드웨어의 정수 데이터패스는 정수 reference와 정확히 0 오차를 일치한다(bit-exact). 한편 float 공든 모델 내부에서 나오는 $err < 5 \times 10^{-16}$ 은 부동소수 reference 자체의 반올림 오차일 뿐 하드웨어 비교값이 아니므로, 둘을 분리해 서술했다. 전체 10,000장은 정수 golden 기준 bit-exact 100%로 확인했다.

6.4 Winograd – 오버클럭 이후: util & 비교

해성 결과 자원 사용량은 LUT 75.88%(48,110/63,400), FF 49.07%(62,221/126,800), **DSP 98.33%(236/240)**, BRAM 41.85%(56.5/135)이다. DSP가 거의 포화 상태를, Winograd multiply array(184) + Conv1 2×(36) + FC(16)가 예상을 빠듯하게 채운다.

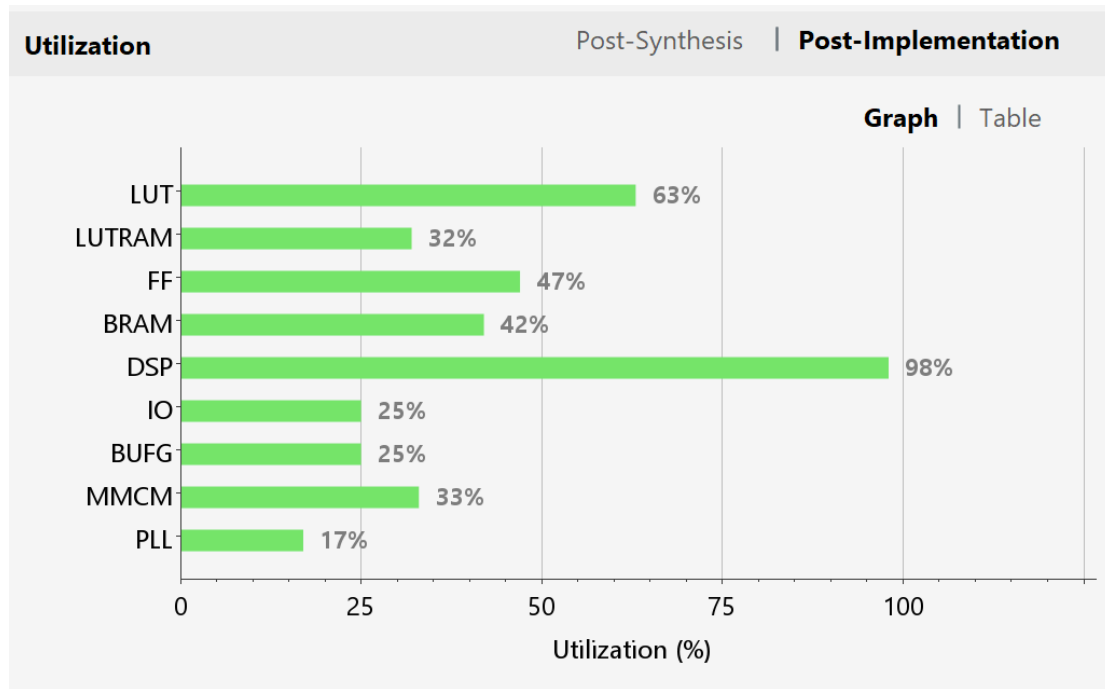


그림 39: 그림: Winograd utilization (DSP 236/240 = 98.33%)

timing은 171.42 MHz에서 **met** (WNS +0.222 ns, 0 failing endpoint)으로 달렸다(서사는 Section 5.2). 다만 보드 실행은 수행하지 못했다(보드 제출 마감 이전에 fallback 명확정). 따라서 정확도·latency는 시뮬레이션 검증 + implementation timing + cycle 추정으로 보고한다: 시뮬레이션 bit-exact(100/100)이므로 보드에서도 baseline과 동일한 10,000/10,000이 기대되며, latency는 $1,341 \times 10, \frac{000}{171.42} \text{ MHz} \approx 78 \text{ ms}$ (compute-only, feed 무시)로 추정된다 — 모두 추정이며 보드 실행이 아님을 명시 한다. (Baseline vs Winograd 종합 비교는 표 T12, Section 5.3.)

7 Discussion

7.1 DSP48E1 SIMD packing의 재평가 — 특히 -128 보정

SIMD packing은 본 설계에서 가장 큰 문제가 컸던 결정이다. 한 DSP가 두 INT8 곱을 처리하므로 같은 240개 예상으로 사실상 두 배의 연산을 얻었고, 이는 Conv2를 192 DSP로 달을 수 있게 한 직접적 전제였다.

-128 corner case 보정은 과대평가하지 않도록 위치를 분명히 둔다. 이론상 $W_1 = -128$ 이고 $W_0 < 0$ 일 때 25비트 표현이 overflow하므로 산술 보정항을 설계에 포함했으나, 본 과제의 실제 제공 가중치에는 -128이 존재하지 않아 이 보정 경로는 실행에서 한 번도 활성화되지 않았다 (Section 2.3 그림 F9의 전수 검증: 세 레이어 모두 -128 cnt=0). 따라서 이 보정을 “실행 성능 기여”로 과대평가해서는 안 된다. 그 가치는 두 가지 다른 측면에 있다. 첫째는 일반성이다 — 임의의 INT8 가중치(재학습된 다른 모델 포함)에 대해서도 bit-exact를 보장하므로, 이 PE는 본 과제에 한정되지 않는 재사용 가능한 빌딩 블록이다. 둘째는 선행연구 대비 위치다 — Xilinx WP486은 27비트 A 포트를 가진 DSP48E2 전용이고 Vestias(FPL’17)는 -128에서 손상이 발생하는데, 본 기법은 더 좁은 DSP48E1(25비트)에서 산술 보정만으로 전 케이스를 무손상 처리한다. ²⁴ exhaustive 검증(Section 6)이 이 일반성을 뒷받침한다. 실제 데이터엔 -128이 없었음을 명시하는 것이 오히려 주장의 신뢰도를 높인다.

7.2 디버깅 사례 1 — AXI-Lite write hang

CSR read는 정상인데 첫 CSR write에서 MicroBlaze가 무한 hang하는 문제가 있었다. 원인은 Xilinx “Create AXI4 Peripheral → Lite” 템플릿의 핸드셰이크 버그였다 — write address(AW)와 write data(W)가 도착하는 순서를 slave FSM이 암묵적으로 가정하고 있어서, W가 AW보다 먼저 오는 경우(AXI 규약상 허용) FSM이 오프 BVALID를 발행하지 못했다. 해결은 `AWVALID && WVALID`가 동시에 성립할 때만 ready를 assert하도록 핸드셰이크를 고친 것이다(코드 C9는 부록).

이 사례의 일반적 교훈은 **AXI interconnect**는 **W**를 **AW**보다 먼저 보낼 수 있으며, **slave**는 채널 도착 순서를 가정해서는 안 된다는 것이다. 벤티 템플릿이라고 해서 모든 행법 시나리오를 처리한다고 믿을 수 없다는 점도 함께 확인했다.

7.3 디버깅 사례 2 – NBA register race

이 버그는 빨간 양상 자체가 교훈적이었다. 단위 테스트(maxpool 단독 40장)는 통과하는데 통합 테스트(conv1→conv2→maxpool 40장)에서 img 1부터 거의 모든 픽셀(~115/144)이 틀렸다. 원인을 추적하니 maxpool의 image 1 write_done이 conv2의 image 1 write_done보다 82 cycle 빨랐다 – 즉 maxpool이 conv2가 아직 c2pool 버퍼에 쓰지 않은 영역을 읽고 있었다.

근본 원인은 NBA(non-blocking assignment) 타이밍이었다. maxpool FSM이 핸드셰이크 카운터

`data_ready =`
prior_diff를 같은 사이클에 갱신(NBA)하면서, 동시에 그 값을 조합 조건 (`prior_diff < 0`) 으로 FSM 전이에 썼다. NBA는 사이클 끝에 갱신되므로 조건은 이전 사이클의 값을 보게 되고, 그 결과 한 박자 이은 잘못된 전이가 일어났다. 단위 테스트가 이를 놓친 이유는, 단독 TB가 입력 pulse를 인위적으로 벌려 주어 race window가 생기지 않았기 때문이다 – engine 간 자연스러운 타이밍에서만 드러나는 결함이었다. 해결은 현재 사이클의 trigger를 반영한 조합값 prior_diff_next로 조건을 판정한 것이다(코드 C10은 부족).

일반적 교훈은 **counter** 기반 조건으로 전이를 결정하는 FSM은, 그 **counter**가 같은 사이클에 갱신될 때 반드시 ***_next** 조합값을 써야 한다는 것이다. 또한 이 사례는 단위 테스트만으로 engine 간 상호작용 결함을 못 잡으므로 통합 테스트가 필수임을 보여준다.

주: Direct/Winograd의 오버클럭 단계별 진단 서사(BMG register, fanout, silent timing failure, reset tree, phys_opt 등)는 Section 5에 있다. 그 과정에서 얻은 일반 교훈(high-fanout net의 route delay가 벽이며 max_fanout 복제가 처방, positive WNS @ slow corner만 신뢰)도 거기에 정리했다.

7.4 Power 분석 (경세 우선순위 #2)

경세는 latency 다음으로 power를 중시하고(p.6), power 리포트 setup을 default에서 바꾸지 말라고 못박는다(p.7). 따라서 power를 단순 수치(Section 6)로 끝내지 않고 트레이드오프 관점에서 해석한다.

먼저 오버클럭과 power의 관계다. 클럭을 150에서 200 MHz로 올리면 dynamic power는 switching 빈도에 비례해 증가한다. 그러나 경세 #2의 기준은 “throughput을 손해 보지 않는 선에서 최소 power”이므로, latency가 실제로 줄어드는 한 클럭 상승은 #2위반이 아니라 #1(latency 최우선)과 정향한다. 더 의미 있는 지표는 단위 추론당 에너지(**energy per inference = power × latency**)다 – 클럭을 올려 latency가 짧아지면 전력이 다소 늘어도 추론 한 건당 에너지는 오히려 줄어든 수 있다.

다음은 Baseline과 Winograd의 power 비교다. Winograd는 DSP 점유가 더 높고(94%→98%) transform network 때문에 LUT가 늘어 정적/동적 power 구성이 다르다. 핵심 질문은 곱셈 수 3.13× 감소(곱셈을 가산으로 치환)가 연산 에너지를 실제로 줄이는지, 아니면 transform 오버헤드가 그 이득을 상쇄하는지다. 이는 측정값으로 평가해야 하며, Winograd는 보드 실측이 없어 implementation power summary만 제시한다(Section 5.2 그림). 모든 수치는 default setup 측정임을 명시해 신뢰성을 담보한다.

7.5 설계 통찰 – 병목의 이동과 분산 제어

이 프로젝트에서 반복적으로 확인된 것은 지엽적 최적화가 전역 병목을 이동시킨다는 점이다. 직접 conv 단계에서는 Conv2(약 1,799 cyc)가 병목이고 Conv1은 1,634 cyc였다. Conv2를 Winograd로 약 1,341 cyc까지 줄이자 이번에는 **Conv1(1,634 cyc)**이 새 병목이 되었다 – Conv2만 빠르게 해서는 전체 throughput이 1,634에 묶인다. 그래서 Conv1을 2× rebalance해 837 cyc로 낮췄고, 그 결과 병목은 다시 Winograd-Conv2(약 1,341 cyc)로 돌아가 파이프라인 floor가 1,634에서 약 1,341로 내려갔다. 즉 두 레이어가 정확히 같아지는 것이 목표가 아니라, 더 느린 쪽을 더 빠른 쪽 아래로 끌어내려 전역 floor를 낮추는 것이 핵심이다.

단계	Conv1 cyc	Conv2 cyc	전역 floor
직접 conv (baseline)	1,634	~1,799	~1,799
Winograd만 적용	1,634	~1,341	1,634
1. Conv1 2× rebalance	837	~1,341	~1,341

표 19: 표 T3: 병목 이동 – 각 단계의 Conv1·Conv2 cyc와 전역 floor

또 하나는 분산 FSM 제어의 이점이다. 중앙 컨트롤러 대신 각 engine이 자체 FSM과 핸드셰이크를 동기하는 구조(Section 3.1.4)는, 제어 신호의 거대 fanout과 그로 인한 timing 부담을 구조적으로 희석한다. Section 5의 timing closure에서 반복적으로 문제가 된 것이 high-fanout 제어 net이었음을 떠올리면, 애초에 중앙 컨트롤러를 피한 결정이 closure를 비교적 수월하게 맞든 한 요인이었다고 평가할 수 있다.

7.6 한계와 Future Work

가장 중요한 한계는 병목이 연산에서 데이터 전송으로 이동했다는 점이다. 클럭을 100에서 200 MHz로 올렸지만 latency는 2배가 아니라 1.72×만 줄었다. 실측 프로파일상 전체 시간의 72%가 100 MHz 도메인의 blocking CDMA 입력 feed에 쓰이기 때문이다(Section 6). 가속기 클럭을 올려도 compute slice만 양측의 feed 시간은 그대로이므로, 설계가 compute-bound에서 memory/feed-bound로 전이한 것이다. 따라서 다음 개선 대상은 연산이 아니라 **feed overlap** (non-blocking/prefetch CDMA + 입력 bank 2개 초과)이다. 이것이 선행되어야 Winograd의 연산 감소가 비로소 end-to-end latency로 환원된다.

둘째, Winograd는 200 MHz를 달지 못하고 171.42 MHz로 fallback했으며, 무엇보다 보드 실측을 수행하지 못했다 – 보드 제출 마감 시점의 시간 제약 때문이다. 따라서 Winograd의 성능은 시뮬레이션 bit-exact + 171.42 MHz timing-met + cycle 추정(~78 ms)으로만 보고되며, 보드 bring-up과 200 MHz closure(Section 5.2의 처방 – route-bound은 over-constrain, logic-bound reduce는 파이프라인 분할 – 을 결함)가 명확한 후속 작업이다.

셋째, Section 7.1에서 보았듯 -128 보정의 가치는 실측 성능이 아니라 일반성 측면에서만 유효하다 – 다른 가중치로 재학습하는 경우에 의미를 갖는다.

8 Conclusion

본 프로젝트는 Arty A7-100T 위에서 MNIST 10,000장을 분류하는 INT8 CNN 가속기를 설계하고, end-to-end latency 최소화를 목표로 최적화하였다. 결과는 두 가래로 정리된다.

검증 완료된 **baseline**. 명세의 INT8 직접 컨볼루션을 비트-정확하게 구현하고 200 MHz로 timing closure(WNS +0.011 ns @ slow corner)하여, MNIST 10,000장에 대해 **10,000/10,000** 정확도 약 **98 ms** latency를 보드에서 실측하였다. 이 과정에서 가장 큰 작업은 -8.6 ns에서 +0.011 ns까지 달아간 timing closure 여정(Section 5)이었으며, 그 핵심 통찰은 이 칩의 타이밍 벽이 호직 길이가 아니라 high-fanout 제어 너트 net의 route delay에 있었고 max_fanout 드라이버 복제가 일관된 처방이었다는 것이다.

제안 알고리즘. Conv2가 곱셈의 90%를 차지한다는 분석에서 출발해, (1) DSP48E1 한 개로 두 INT8 곱을 처리하는 SIMD packing, (2) 곱셈을 직접 conv 대비 3.13× 줄이면서 INT8 bit-exact를 유지하는 complex Winograd F(4×4, 3×3), (3) 그로 인해 이동한 병목을 맞추는 Conv1 2× rebalance를 도입하였다. Winograd 데이터패스는 시뮬레이션에서 bit-exact로 검증되었고 implementation은 **171.42 MHz로 timing-met** (200 MHz는 두터지 장기식 병목 연쇄로 미달)이다. 다만 보드 제출 마감 시점의 시간 제약으로 보드 실측을 수행하지 못해, 1만 장 latency는 cycle 추정(~78 ms @171.42 MHz)으로만 보고한다.

마지막으로, 본 설계의 다음 한계는 연산이 아니라 데이터 전송에 있다. 클럭을 두 배로 올려도 latency가 1.72×만 줄어든 것은 전체 시간의 72%가 PS-PL feed에 묶여 있기 때문이며, 따라서 Winograd의 연산 이동은 온전히 얻으려면 feed overlap이 선행되어야 한다 – 이는 향후 작업의 분명한 방향이다.

References

- [1] EEE351 Intelligent System Design and Application, Assignment: CNN Accelerator (과제 명세서).

- [2] A. Lavin and S. Gray, “Fast Algorithms for Convolutional Neural Networks,” *Proc. IEEE CVPR*, pp. 4013–4021, 2016. (Winograd minimal filtering, $F(m,r)$)
- [3] A. L. Toom, “The Complexity of a Scheme of Functional Elements Realizing the Multiplication of Integers,” *Soviet Mathematics Doklady*, vol. 3, pp. 714–716, 1963. (다항식 곱셈 기법 곱셈; 원본 *Doklady Akad. Nauk SSSR*, 150(3):496–498)
- [4] S. A. Cook, *On the Minimum Computation Time of Functions*, Ph.D. thesis, Harvard University, 1966. (Toom–Cook 알고리즘 정식화)
- [5] AMD (Xilinx), *Vivado Design Suite 7 Series FPGA Libraries Guide (UG953)* – DSP48E1 primitive. <https://docs.amd.com/r/en-US/ug953-vivado-7series-libraries/DSP48E1> (SIMD weight packing의 DSP48E1 포트·연산 참조)
- [6] Xilinx, *7 Series DSP48E1 Slice User Guide (UG479)*. (25×18 multiplier 구조)
- [7] Xilinx, *Convolutional Neural Network with INT4 Optimization on Xilinx Devices (WP521)*. (DSP48E2 INT8/INT4 packing – 본 기법과 대비)
- [8] M. Véstias et al., “A Configurable Architecture for Running Hybrid Convolutional Neural Networks in Low-Density FPGAs,” *Proc. FPL*, 2017. (저밀도 FPGA CNN, INT8 packing 비교 대상)
- [9] EEE351 Week 9 강의 자료 – AXI4-Lite CSR slave 설계 참고.
- [10] PyTorch – golden reference 모델 및 INT8 양자화 파라미터.

※ 인용 형식·서지정보는 제출 전 최종 확정한다.

Appendix A. 코드 뺄체 (본문에서 참조)

본문에 인용된 코드 뺄체 중 본문에 직접 실지 않은 것을 모은다(C1·C2·C4·C5는 본문 §3·§5에 있음).

C3 — **kcol_accumulator.v**: 커널 역 3개를 3사이클에 걸쳐 24-bit 누적 (Section 2.4.3).

```
always @(posedge clk) begin
  if (rst) begin out <= 24'sd0; out_valid <= 1'b0; end
  else case (phase) // 0=first, 1=middle, 2=last
    2'd0: out <= in; // first : reset + 첫 누적
    2'd1: out <= out + in; // middle: 누적
    2'd2: begin out <= out + in; out_valid <= 1'b1; end // last: 누적 + valid
  endcase
end
```

C6 — **main.c**: 10,000쌍 inference loop (뺄대 bank preload + start + done poll) (Section 3.2.7).

```
for (i = 0; i < N_IMG; i++) {
  bank = (i & 1) ? 0x400 : 0x000; // ping-pong: 뺄대 bank에 preload
  memcpy(IMEM_BASE + bank, images[i], 784);
  *CTRL = START_BIT; // start pulse
  while (!(*STATUS & DONE_BIT)) ; // done poll
  result[i] = (*STATUS >> 1) & 0xF; // 4-bit class
}
```

C7 — **wino_truncate.v**: Winograd 1/16 + layer >>10 을 곱셈한 >>>14 + sat + ReLU (Section 4.1.1).

```
wire signed [YW-1:0] y_i = $signed(y16_flat[i*YW +: YW]);
wire signed [SW-1:0] sh_i = y_i >>> SHIFT; // SHIFT=14 (=4 winograd + 10 layer)
// 이후 saturate(±127) + ReLU → 8-bit 출력 (truncate_relu 와 동일 규칙)
```

C9 — AXI-Lite write hang 수정: AWVALID && WVALID 동시 조건 (Section 7.2).

```
// W 가 AW 보다 먼저 와도 안전: 둘 다 valid 일 때만 ready assert
assign awready = ~axi_bvalid & awvalid & wvalid;
assign wready = ~axi_bvalid & awvalid & wvalid;
```

C10 — NBA register race 수정: **prior_diff_next** 조합값으로 판정 (Section 7).

```
// (수정 전) data_ready = (prior_diff < 0) ← 이전 사이클 stale 값
// (수정 후) 현재 사이클 trigger 를 반영한 조합값 사용
wire signed [2:0] prior_diff_next =
  prior_diff + (rdone ? 3'sd1 : 3'sd0) - (prior_wdone ? 3'sd1 : 3'sd0);
wire data_ready = (prior_diff_next < 3'sd0);
```

Appendix B. CSR Memory Map & 첨부물

CSR(AXI-Lite slave, `csr_axi.v`)는 PL 인터페이스로 `enable / start / img_ready` (PS→PL), `img_done / input_consumed` (PL→PS)와 timer 레지스터를 노출한다. 정확한 주소·비트 필드는 `csr_axi_slave_lite_v1_0_csr.v`와 대조해 확정한다(표 T2).

첨부 제출물. 전체 Vivado Block Design은 `archive/v2_overclock_200MHz_backup/`의 `cnn_accel_bd_200MHz.pdf`(붙측도)와 `cnn_accel_bd_200MHz.tcl`(재현용 TCL)로 보고서와 함께 제출한다. data-path BMG 파라미터(Port A 32b / Port B 8~128b)는 `docs/ip_spec/`의 BMG 스펙·스크린샷 참조.